

Lightweight CNN Model for Real-Time Rice Leaf Disease Detection on Mobile Devices

By

Md. Nazibur Rahman
0242220005101410

Md Shah Alam
0242220005101409

FINAL YEAR DESIGN PROJECT REPORT

This Report Presented in Partial Fulfillment of the
Requirements for the **Degree of Bachelor of Science in**
Computer Science and Engineering

Supervised by

Dr. Md Alamgir Kabir
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

Co-Supervised by

Shah Md. Tanvir Siddiquee
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University



DAFFODIL INTERNATIONAL UNIVERSITY
Dhaka, Bangladesh

September 09, 2026

APPROVAL

This Project titled “**Lightweight CNN Model for Real-Time Rice Leaf Disease Detection on Mobile Devices**” submitted by **Md. Nazibur Rahman** and **Md Shah Alam** to the Department of Computer Science and Engineering, Daffodil International University, has been accepted as satisfactory for the partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Engineering and approved as to its style and contents. The presentation has been held on **09-09-2026**.

BOARD OF EXAMINERS

Name

Board Chairman

Designation,

Department of CSE, FSIT

Daffodil International University

Name

Internal Examiner 1

Designation,

Department of CSE, FSIT

Daffodil International University

Name

Internal Examiner 2

Designation,

Department of CSE, FSIT

Daffodil International University

Name

External Examiner

Designation,

Department of CSE, FSIT

Daffodil International University

DECLARATION

We hereby declare that this project has been done by us under the supervision of **Dr. Md Alamgir Kabir, Assistant Professor**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project or any part of this project has been submitted elsewhere for the award of any degree or diploma.

Supervised by:

Dr. Md Alamgir Kabir

Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

Co-Supervised by:

Shah Md. Tanvir Siddiquee

Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

Submitted by:

Md. Nazibur Rahman

Student ID: 0242220005101410
Department of Computer Science and Engineering
Daffodil International University

Md Shah Alam

Student ID: 0242220005101409
Department of Computer Science and Engineering
Daffodil International University

ACKNOWLEDGEMENTS

This work would not have been possible without the support and contributions of many individuals throughout the past three semesters. We are deeply grateful to everyone who has assisted and supported us in one way or another during the completion of this project.

First, we express our heartfelt thanks and gratitude to the Almighty Allah for His divine blessings, which made it possible for us to successfully complete our **Final Year Design Project (FYDP)**.

We would like to express our profound gratitude and indebtedness to **Dr. Md. Alamgir Kabir, Assistant Professor**, Department of Computer Science and Engineering, Daffodil International University, Dhaka, Bangladesh, for his valuable supervision and guidance throughout this project. His deep knowledge and keen interest in the field of **Artificial Intelligence, Deep Learning, and Computer Vision** provided us with the opportunity to carry out this project successfully. His endless patience, scholarly guidance, continuous encouragement, constant supervision, constructive criticism, valuable advice, and careful review of our work at every stage played a vital role in completing this project.

We would like to express our heartfelt gratitude to the **Head of the Department of Computer Science and Engineering**, Daffodil International University, for his kind support and assistance in completing our project. We are also grateful to all the faculty members and staff of the Department of Computer Science and Engineering for their valuable support throughout our academic journey.

We would also like to thank our course-mates and friends at Daffodil International University who participated in discussions, shared their ideas, and provided valuable support during the development and completion of this project.

Finally, we express our deepest gratitude to our parents and family members for their unconditional support, patience, encouragement, and continuous prayers throughout our academic journey and during the completion of this project.

ABSTRACT

Leaf diseases like Bacterial Blight, Blast, Brown Spot and Tungro are one of the most important diseases affecting the productivity of rice, which is one of the most important food crops in the world. The traditional diagnosis methods are based on visual assessment of the farmer or agricultural specialist, which are difficult to perform, particularly in a resource-poor environment, and may be time consuming, inconsistent, and inaccessible. Deep learning models have been used successfully for automated disease classification tasks, but most of the current models are complex and difficult to implement on mobile devices. In this paper, a lightweight Convolutional Neural Network (CNN) for real-time rice leaf disease classification on mobile platforms is proposed. The proposed model contains depthwise-separable convolutions, Squeeze-and-Excitation attention, residual feature propagation and HardSwish activation to facilitate an efficient classification accuracy/computation cost trade-off. The dataset consisted of 5 classes (Bacterial Blight, Blast, Brown Spot, Healthy and Tungro) with 4,212 images, which were acquired from two public image repositories and passed through a deterministic preprocessing pipeline. The proposed model has only 0.220 million parameters and a model size of approximately 0.84 MB, it require 83.61 million MACs and achieve a test accuracy of 99.77% with an average inference time of 2.293 ms (436.10 FPS) when using the standardized benchmarking. The model trained was converted to the TensorFlow Lite (TFLite) format and implemented in a Flutter application for Android, which allowed for complete on-device inference without internet access. The experimental results on MobileNetV2, MobileNetV3-Small, MobileNetV3-Large, EfficientNet-B0 and GhostNet show that the proposed model achieves excellent classification accuracy while reducing the number of parameters, model size and inference time. These findings show the potential of practical application of high accuracy, low-computation disease detection systems for mobile devices with limited resources in the agricultural sector.

Table of Contents

Approval	i
Declaration	ii
Acknowledgements	iii
Abstract	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Introduction.....	1
1.2 Motivation	2
1.3 Objectives	2
1.4 Methodology	2
1.5 Project Outcome	3
1.6 Organization of the Report	3
2 Background	4
2.1 Introduction.....	4
2.2 Literature Review	5
2.2.1 Similar Applications	7
2.3 Gap Analysis	8
2.4 Summary	10
3 Research Methodology	11
3.1 Methodology	11
3.1.1 Overview	11
3.1.2 Proposed Methodology	12
3.1.3 Functional and Nonfunctional Requirements	13
3.1.4 Data Flow Diagram	14
3.1.5 UI Design	14
3.2 Detailed Methodology and Design	15
3.3 Project Plan	27
3.4 Task Allocation.....	27
3.5 Summary	28

4	Implementation and Results	29
4.1	Environment Setup.....	29
4.2	Testing and Evaluation.....	29
4.3	Results and Discussion	37
4.4	Summary	37
5	Engineering Standards and Design Challenges	39
5.1	Compliance with the Standards	39
5.1.1	Software Standards.....	39
5.1.2	Hardware Standards	40
5.1.3	Communication Standards	40
5.2	Impact on Society, Environment and Sustainability	40
5.2.1	Impact on Life.....	40
5.2.2	Impact on Society & Environment.....	41
5.2.3	Ethical Aspects	41
5.2.4	Sustainability Plan.....	41
5.3	Project Management and Financial Analysis.....	42
5.4	Complex Engineering Problem	43
5.4.1	Complex Problem Solving	43
5.4.2	Engineering Activities	45
5.5	Summary	45
6	Conclusion	47
6.1	Summary	47
6.2	Limitation	47
6.3	Future Work	48
	References	49

List of Figures

3.1	Proposed Research Methodology Workflow	12
3.2	Data Flow Diagram of the Proposed System.....	14
3.3	User Interface of the Mobile Application	15
3.4	Distribution of the Final Rice Leaf Disease Dataset	16
3.5	Image Preprocessing Pipeline	17
3.6	Original and Preprocessed Image.....	19
3.7	Overall Architecture of the Proposed Model	19
3.8	Internal Architecture of the Proposed Block	20
3.9	Squeeze-and-Excitation Attention Integrated into the Proposed Block ..	21
4.1	Training and validation accuracy vs loss curves of the models	30
4.2	Confusion matrices of the evaluated models	33
4.3	Grad-CAM and Grad-CAM++ visualizations of the Proposed Model	36

List of Tables

2.1	Summary of Related Works in Rice Leaf Disease Detection.	5
2.2	Comparison of Similar Applications.....	7
2.3	Gap Analysis of Related Works.	8
3.1	Functional Requirements	13
3.2	Non-Functional Requirements.....	13
3.3	Final Dataset Distribution.	16
3.4	Proposed Model Architecture.....	20
3.5	Training Configuration.....	23
3.6	Project Plan.....	27
3.7	Task Allocation.....	27
4.1	Dataset and Experimental Environment Summary.....	29
4.2	Overall Classification Performance.....	32
4.3	Computational Performance Comparison.	34
4.4	Inference Performance Comparison.	34
5.1	Estimated Project Budget.	42
5.2	Alternative Low-cost Budget.....	42
5.3	Mapping with Complex Engineering Problem.	43
5.4	Mapping with knowledge Profile.....	44
5.5	Mapping with Complex Engineering Activities.	45

Chapter 1

Introduction

In this chapter, a background and context of the project are presented, followed by the motivation for developing a lightweight rice disease detection system and the objectives of the work. It also gives a brief summary of the methodology adopted, main findings of the project and the structure of the subsequent chapters.

1.1 Introduction

Rice is one of the most crucial food crops and contributes greatly to food security and agricultural production. Rice is prone to several diseases, which may negatively affect plant health and productivity in rice. Rice leaves may show signs of disease caused by Bacterial Blight, Blast, Brown Spot, and Tungro and can result in substantial crop loss, if the diseases are not diagnosed and treated properly [1][2].

Rice diseases are traditionally diagnosed using visual assessment by farmers or the agricultural experts. While many of the symptoms of disease may be recognized by an experienced person, manual diagnosis can be time-consuming, and requires the knowledge and experience of the person making the diagnosis. Furthermore, the symptoms of the disease and environmental differences may create difficulties in diagnosing [40].

Artificial Intelligence (AI) and Deep Learning (DL) offers new opportunities for automated plant disease detection. This has led to the development of Convolutional Neural Networks (CNNs) which have proven to be powerful tools in the field of disease classification using images, due to their ability to automatically form features that are relevant to the problem from the input images. CNN applied method has been used for some studies in rice disease classification and obtained good results [1][2][11][12].

However, many of the deep learning models are very computation intensive, demanding lots of memory and storage. These requirements may pose challenges in deploying these models to resource-limited devices like smart phones. Therefore, lightweight CNN architectures are emerging as a crucial research stream for achieving an appropriate classification accuracy / computational cost trade-off [4][5][6][7].

The main objective of this project is to design a lightweight CNN-based deep learning rice leaf disease classification model. The proposed model aims to classify rice leaf images into five classes - Bacterial Blight, Blast, Brown Spot, Healthy, and Tungro without compromising on the size of the architecture, which is an important factor to consider when implementing the model in practice.

The project additionally expands the classification model to a mobile solution. The trained model is then converted to the mobile-friendly TensorFlow Lite format and embedded within a Flutter application for the Android platform to enable rice leaf

images to be analyzed directly on the mobile device. The final system thus not only solves the problem of good classification of the disease by using machine learning, but also the problem of minimizing the model size and hence reducing the computational requirements for deployment on mobile devices.

The proposed model successfully achieves the test accuracy of 99.77%, and the model size is about 0.84 MB with only 0.220 million parameters. The outcomes show the capability of a lightweight CNN-based methodology for correctly categorizing rice diseases with a small computational load.

1.2 Motivation

The main goal of this project is to develop an automatic rice disease detection system that can achieve high classification accuracy with low computing power. Though deep learning models have proven to be effective in the classification of plant diseases, in practice they require a substantial amount of computational and memory resources that may hinder their application on resource-limited devices [4][5][6][7].

Automated disease detection using mobile-based solution will make this more accessible as it will enable the user to analyze rice leaf images with a smartphone. This is especially helpful if the expertise or high performance computing power of an agricultural researcher is unavailable.

Another significant reason is the need of accuracy and efficiency. Too complex a model may not be practical for deployment on a mobile device, and too simple may not be able to differentiate between diseases that look alike. Thus, a lightweight model with high classification accuracy is desired, which has less computational and memory consumption.

1.3 Objectives

- 1) To design and develop a lightweight CNN model for accurate rice leaf disease classification.
- 2) To compare the proposed model with some of the existing lightweight CNN architectures.
- 3) Building a mobile app to detect diseases real time on the device.

1.4 Methodology

The project has a sequential procedure, starting with the preparation of the data sets and ending with the mobile deployment. Images of rice leaves belonging to the five classes are prepared and undergo some pretreatment before they are utilized for model training. During the previous phase, the pre-processing pipeline is established that involves image resizing and enhancement operations for appropriate input to the classification model.

The proposed light weight CNN model is then trained and tested with the suitable classification metrics. The value of the model is evaluated post training for its predictive capability as well as its computational efficiency. The accuracy, precision,

recall, and F1-score are used for measuring classification performance, while model size, MACs, FLOPs, memory usage, and inference time are used for measuring computational characteristics.

The trained model is then transformed into TensorFlow Lite format and used in an Android app built with Flutter.

1.5 Project Outcome

This project led to the development of a lightweight rice disease classification CNN model and a usable mobile app for real-time disease detection.

With the test set of 427 samples, the proposed model was able to achieve 99.77% test accuracy. It has 220165 parameters, size of the model is ~0.84 MB, and it demands ~83.61 million MACs and ~167.22 million FLOPs.

It has been successfully converted to the TensorFlow Lite (TFLite) and added to a Flutter-based Android app. A completed application can be given a rice leaf image and return the disease category prediction along with a confidence score.

Hence, the project has been able to integrate the correct classification of rice disease, light-weight model design, computational analysis, and mobile deployment in a single system.

1.6 Organization of the Report

The rest of this report is divided into five chapters. The background study and literature review for rice diseases, deep learning, CNNs, lightweight architectures, attention mechanisms, explainable AI, and mobile deployment are presented in Chapter 2, and the research gap identified. The research methodology is discussed in Chapter 3, which involves data collection, data preprocessing, proposed architecture of the model, training and validation process of model, metrics used for the evaluation and deployment process. The implementation and experimental results such as classification performance, computational analysis, explainability analysis, model comparisons, and mobile application results are provided in Chapter 4. The engineering standards and design challenges, as well as societal and environmental issues, ethical concerns, sustainability and other engineering aspects of the project are discussed in chapter 5. Finally, Chapter 6 presents a summary of the important results, constraints, and suggested future research directions.

Chapter 2

Background

The background knowledge and related research on rice disease detection using deep learning are presented in this chapter. It summarizes the existing CNN-based and light-weight methods, and discusses the computational efficiency, attention mechanisms, feature enhancement, explainability, and practical deployment before elaborates the research gap which our work solved.

2.1 Introduction

Rice is one of the significant crops in agriculture and the diseases of rice leaves can adversely impact on plant health and productivity. The common method of disease recognition is mostly visual and relies on the expertise and experience of the farmer or agricultural specialist, and can be time consuming. Different disease symptoms and environmental factors can also make it difficult to make correct identification. Developing image-based automated approaches has been shown to be a possibility by Deng et al. (2021) [40] in the automatic diagnosis of rice diseases using deep learning.

In the field of plant disease classification using image analysis, deep learning (including Convolutional Neural Networks (CNNs)) has emerged as a significant technique. CNNs can be trained directly from leaf images to learn the discriminative visual features, thereby avoiding the need for manually designed visual features. The study of the deep CNN approach with an improved dataset for the classification of rice leaf diseases was conducted by Rawat et al. (2023) [1] and Hasan et al. (2024) [2].

While powerful deep learning models can be used for robust classification, they might be too computationally intensive and demand too much memory to be used in resource-constrained devices. This has motivated the study of lightweight architectures to lower the model complexity while still ensuring good classification ability. Different lightweight techniques for disease detection and classification were explored by Pan et al. (2025) [4] and Sirisha et al. (2026) [5] and Jing (2026) [6] and Bhagat et al. (2024) [7].

Moreover, research has recently focused on attention mechanisms and feature-enhancement techniques to boost the representation ability of light networks. CNN and attention-based methods were explored in recent reviews by Carlson et al. (2025) [8] and Tasci (2026) [10] which added attention mechanisms to lightweight architectures, and Zhang et al. (2025) [14] and Xu et al. (2025) [20] did the same.

Practical deployment is also an important factor to consider. Kayne et al. (2024) [9] and Velasco et al. (2025) [13] studied the mobile classification of rice diseases, as well as the optimization and deployment of models on platforms with limited resources, such as edge platforms (Martins et al., 2025) [15] and Zheng et al., 2025) [17] and Xan et al., 2024) [39].

The background of this study is therefore focused on designing a disease classification method which not only has high prediction performance, but also has low model size, low complexity, high efficiency of model inference and easy extension to mobile deployment.

2.2 Literature Review

Research on automated rice disease detection has developed from conventional deep learning-based classification toward lightweight and deployment-oriented approaches. Table 2.1 summarizes the relevant literature reviewed in this research.

Table 2.1: Summary of Related Works in Rice Leaf Disease Detection

Author	Year	Title / Model	Methodology	Key Findings
Rawat et al. [1]	2023	Rice Leaf Diseases Classification Using Deep Learning Techniques	Deep learning / CNN-based classification	Demonstrated the applicability of deep learning techniques for automated rice leaf disease classification.
Hasan et al. [2]	2024	Rice Leaf Disease Detection Using dCNN and Enhanced Dataset	Deep CNN + enhanced dataset	Demonstrated the effectiveness of deep CNN-based disease detection and highlighted the importance of dataset enhancement.
Mahin et al. [3]	2026	Custom CNN and EfficientNet-B0	Custom CNN + EfficientNet-B0	Reported 99.12% accuracy and 99.63% F1-score with approximately 4.8M parameters.
Pan et al. [4]	2025	SSD-YOLO	Lightweight object-detection network	Reported 95.19% precision, 95.32% mAP@0.5, and 6.08 MB model size.
Sirisha et al. [5]	2026	Lightweight Residual Dilated CNN-Transformer	CNN-Transformer hybrid	Investigated a lightweight residual dilated CNN-Transformer framework for efficient rice leaf disease classification.
Jing [6]	2026	Lightweight Deep Learning Model for Mobile Devices	Lightweight deep learning	Focused specifically on crop disease detection on mobile devices, emphasizing computational efficiency for mobile applications.
Bhagat et al. [7]	2024	Lite-MDC	Lightweight CNN architecture	Reported 94.14% accuracy, approximately 2.2M parameters, and 33 FPS for real-time plant disease detection.
Kayne et al. [9]	2024	Enhanced CNN for Mobile Applications	Enhanced CNN	Reported 99.12% accuracy with approximately 4.8M parameters for mobile-oriented rice disease classification.
Tasci [10]	2026	DBLA-MobileNetV2	MobileNetV2 + dual-branch lightweight attention	Reported 98.30% test accuracy and evaluated real-time performance on edge hardware.

Velasco et al. [13]	2025	CNN-Based Mobile Application	CNN + mobile application	Reported 96.49% accuracy, 95.81% precision, 96.43% recall, and 96.11% F1-score.
Zhang et al. [14]	2025	Dual-Attention MobileViT-DAP	MobileViT + dual attention	Reported 99.61% accuracy, 0.75M parameters, and 0.23G FLOPs using field-captured images.
Martins et al. [15]	2025	RiceLeafClassifier-v1.0	Quantized deep learning model	Reported 94% accuracy for the quantized model with approximately 2.28 ± 0.35 s/image inference time.
Zheng et al. [17]	2025	MobileNetV2 + DIST + FPGA	MobileNetV2 + knowledge distillation + FPGA	Reported 97.41% accuracy, 43 ms inference time, and approximately 3.2 W power consumption.
Xu et al. [20]	2025	LiSA-MobileNetV2	MobileNetV2 + SE + Swish	Reported 95.68% accuracy, 575.06K parameters, and 331.27M FLOPs.
Cui et al. [21]	2026	LMRNet	Lightweight CNN	Reported 99.26% model accuracy, 92.41% mobile accuracy, and 22.47 FPS.
Duhan et al. [26]	2025	RTR_Lite_MobileNetV2	Lightweight MobileNetV2	Reported up to 99.69% accuracy with approximately 1.05M parameters.
Deng et al. [40]	2021	Automatic Diagnosis of Rice Diseases Using Deep Learning	Deep learning-based diagnosis	Established the feasibility of automatic rice disease diagnosis using deep learning.

Discussion of Reviewed Studies

Deng et al. [40] proved the effectiveness of deep learning and CNN-based methods in automated rice disease classification, Rawat et al. [1] also demonstrated their usefulness in rice disease classification, and Hasan et al. [2] showed their effectiveness in the classification of rice diseases using deep learning. These works serve as the basis to leverage image based deep learning instead of relying on manual disease identification.

Later work has largely concentrated on improving the efficiency of the computation. Mahin et al. (2026) [3] studied a hybrid model of a custom CNN and EfficientNet-B0, and Pan et al. (2025) [4] and Sirisha et al. (2026) [5] tested the lightweight detection approach and CNN-Transformer approach respectively. The work in [6] focuses on mobile-based disease detection and [7] explores real-time lightweight disease recognition.

Another line of research in the literature is the attention-based feature enhancement. Attention-based methods were discussed by Carlson et al. (2025) [8] and dual-branch lightweight attention was added to MobileNetV2 by Tasci (2026) [10]. The dual attention in MobileViT [14] is used by Zhang et al. and the SE attention in MobileNetV2 [20]. Zhang and Yu (2025) [38] also investigated the topic of adaptive attention and cross-layer feature fusion.

A number of studies have been conducted on how to get the disease recognition to a platform that has limited resources, which are deployment oriented. Kayne et al.

(2024) [9] and Velasco et al. (2025) [13] looked at mobile apps, and Martins et al. (2025) [15] at quantization. Zheng et al. (2025) [17] used knowledge distillation and FPGA acceleration, while Xan et al. (2024) [39] explored the possibility of an ARM-M microcontroller-based edge solution.

Field-based recognition and real-time systems have also been the subject of research. Zhang et al. (2025) [14] used field-captured images, and Zhang et al. (2025) [24] studied field disease detection under complex conditions. Cui et al. (2026) [21], Zhang et al. (2026) [18], and Yuan et al. (2026) [30] explored lightweight and real-time recognition approaches.

There have been other studies on related tasks and on more general optimization strategies. Gan et al. (2025) [16] and Li et al. (2025) [36] were dedicated to the task of disease detection with YOLO and Li et al. (2025) [19] to lightweight disease segmentation. Upadhyay and Prasad (2025) [28] proposed a light-weight transformer solution and Pi et al. (2026) [33] proposed frequency-aware feature learning.

Explainability has also been a focus of attention. To assess deep learning models, Kondaveeti and Simhadri (2025) [25] used qualitative and quantitative explainable AI techniques to show that it could be valuable to complement the standard performance assessment.

In general, it can be concluded that the reviewed literature has shown that the development of rice disease detection has not only focused on getting the highest classification accuracy, but has also progressed in other ways. The focus of current research is increasingly shifting to lightweight architecture, attention, computational efficiency, real-time inference, field conditions, explainability, and deployment constraints.

2.2.1 Similar Applications

Several works and deployment oriented studies have been conducted to study the application of deep learning in real world rice leaf disease detection on mobile and resource constrained devices. These works are of particular interest to the proposed system as they not only take classification performance into account but also the practicalities of mobile or edge deployments.

Table 2.2: Comparison of Similar Applications

Author	Year	Application / Model	Deployment Focus	Key Findings
Kayne et al. [9]	2024	Enhanced CNN for Mobile Applications	Mobile-oriented rice disease classification	99.12% accuracy with approximately 4.8M parameters
Jing [6]	2026	Lightweight Deep Learning Model	Crop disease detection on mobile devices	Focused on computational efficiency for mobile applications
Velasco et al. [13]	2025	CNN-Based Mobile Application	Mobile application for rice crop disease detection	96.49% accuracy, 95.81% precision, 96.43% recall, and 96.11% F1-score

Martins et al. [15]	2025	RiceLeafClassifier-v1.0	Quantized model and edge deployment	94% accuracy with approximately 2.28 ± 0.35 s/image inference time
Proposed RiceDT	2026	Lightweight CNN + TFLite	Android mobile application with on-device inference	99.77% accuracy, 0.220M parameters, 0.84 MB model size

The existing applications illustrate various ways to get automated plant or rice disease closer to mobile and edge deployment. Kayne et al. [9] targeted a mobile CNN that achieves high classification accuracy, while Jing [6] prioritized low computation for mobile devices. Velasco et al. [13] showed the implementation of CNN classification of rice disease in a mobile app. In addition to the above, Martins et al. [15] were also interested in reducing resource usage for disease detection by considering the quantization of models and deployment of models on the edges.

The proposed RiceDT application is a similar practical approach, but with TensorFlow Lite and Android deployment as well as a lightweight CNN architecture. The proposed model achieved a 99.77% accuracy, only 0.220 million parameters with a model size of 0.84 MB. The model was then converted to TFLite and then embedded in the Android app, which is based on Flutter, for inference to happen directly on the Android device.

2.3 Gap Analysis

The analyzed studies show that there is a significant improvement in the automated rice disease detection. But the studies have vastly different goals, data sets, architectures, evaluation measures, and environments. The comparison below is therefore provided to help to define directions for future research and limitations in the models reviewed rather than to rank the models.

Table 2.2: Gap Analysis of Related Works

Feature / Consideration	Conventional CNN Studies [1][2][40]	Lightweight Models [3][4][5][6][7]	Attention-Based Models [10][14][20][38]	Mobile / Edge Studies [9][13][15][17][39]	Proposed Model
Rice leaf disease classification	Yes	Yes	Yes	Yes	Yes
Lightweight architecture	No	Yes	Yes	Yes	Yes
Attention-based feature enhancement	No	Partial	Yes	Partial	Yes
Feature propagation / skip connection	Partial	Partial	Partial	Partial	Yes

Compact parameter count	No	Yes	Yes	Yes	Yes
Model size evaluation	Partial	Yes	Yes	Yes	Yes
MACs / FLOPs evaluation	No	Yes	Yes	Partial	Yes
Inference-time evaluation	No	Partial	Partial	Yes	Yes
Mobile deployment	No	Partial	Partial	Yes	Yes
On-device inference	No	Partial	No	Partial	Yes
Explainability analysis	Partial	No	No	No	Not evaluated
Practical application	Partial	Partial	Partial	Yes	Yes

The studies reviewed show that lightweight architectures are able to decrease the number of computations that need to be performed, and attention and feature-enhancement mechanisms can increase the representation power of features. Mobile and edge-based studies also show the flexibility of the disease detection system deployment outside traditional computing systems.

But the approaches reviewed in the paper vary in the extent to which they assess these issues. A few studies present classification results without a detailed mobile deployment evaluation and a few others rely on edge hardware like FPGA or Jetson platforms. For instance, Zheng et al. (2025) [17] tested the accelerated performance of an FPGA, and Tasci (2026) [10] tested the performance on the Jetson Nano.

Likewise, some lightweight methods claim lower counts of parameters, but do not explicitly assess inference on smartphones. Even if the number of parameters is too small to express, Xu et al. (2025) [20] reported 575.06K parameters and 331.27M FLOPs, and did not examine the direct inference of smartphones.

Hence, the potential research opportunity is to explore lightweight CNN-based model which is not only capable of classing disease but also requires minimal computation and can be easily implemented on smartphones. This work compares the proposed model with the classification metrics, along with the number of parameters, model size, MACs, FLOPs, inference time, FPS and the usage of GPU memory. The model is also expanded

to support TensorFlow Lite and embedded within a mobile app.

It is not intended to assert the superiority of the approach over previous studies as these would include varying sets of data, experimental environments, hardware, and evaluation procedures. Rather, it is intended that a full evaluation of a lightweight model be made within the confines of its own experiment.

2.4 Summary

The current chapter summarized the research that has already been conducted in the area of automated rice disease detection via deep learning. A total of 17 representative studies were reviewed, which included conventional CNN-based classification, model optimization, real-time detection, mobile/edge deployment, lightweight architectures, and attention mechanisms.

It has been reported in the literature that in recent years, along with the classification performance, computational efficiency and, above all, the feasibility of the application have been highlighted. But the datasets, hardware, architectures and evaluation protocols used in the various studies differ making direct performance ranking between various studies inappropriate.

From the identified research gap, this work aims to design a proposed CNN-based model which takes into consideration the classification accuracy, computational complexity, inference efficiency, and usability in deployment of the model on a smartphone. The method of developing and testing the proposed system is discussed in the next chapter.

Chapter 3

Research Methodology

In this chapter, the methodology adopted to develop, evaluate, and deploy the proposed lightweight CNN-based rice leaf disease classification system is discussed. The methodology is discussed in detail on collection and integration of dataset, data cleaning, image preprocessing, splitting of dataset, model selection, proposed model designing, training configuration, performance testing, explainability analysis and integration of mobile applications. The functional and non-functional requirements, data flow, user interface design, project plan and task allocation are also included.

3.1 Methodology

The research methodology proposed was based on developing a system for classification of rice leaf disease with a light weight deep learning model with an emphasis on classification accuracy and computational efficiency. A structured pipeline has been adopted from the preparation of the rice leaf image dataset to the deployment of the trained model in a mobile application.

The research takes into account five classes: Bacterial Blight, Blast, Brown Spot, Healthy and Tungro. This entire process comprises of data cleansing, deterministic image pre-processing, partitioning of the dataset, model training, comparative evaluation, explainability analysis, and mobile deployment. Special care was taken to ensure that the computational requirements of the proposed model were minimal in order to make it feasible on resource-limited systems.

3.1.1 Overview

The methodology begins with data collection from existing publicly available datasets, followed by data integration, quality screening, duplicate removal, image preprocessing, dataset splitting, model selection, model training, evaluation, explainability analysis, and deployment. The disease images were obtained from the Rice Leaf Disease Image Samples dataset published by P. K. Sathy on Mendeley Data [41], which contains images of Bacterial Blight, Blast, Brown Spot, and Tungro. The Healthy class images were obtained from the Rice Leaf Disease DataSet published by M. M. Hasan on Kaggle [42].

The images from these two sources were integrated to construct the five-class dataset used in this research. The initial integrated dataset contained 7,017 images. Manual quality screening was subsequently performed to remove blurry or unusable images, followed by hash-based duplicate detection and removal. After these cleaning procedures, the final dataset contained 4,212 images.

The cleaned data set was then preprocessed with a fixed image pre-processing pipeline. Center cropping, resizing, contrast enhancement, white balancing, bilateral filtering, HSV-based color enhancement, final resizing, tensor conversion, and ImageNet normalization are the pre-processing steps performed. Processed data set was split into the training, validation and testing sets in the ratio of 80:10:10 roughly.

Then a few popular lightweight CNN architectures were chosen: MobileNetV2, MobileNetV3-Small, MobileNetV3-Large, EfficientNet-B0, and GhostNet. These models were tested under a common classification task with the proposed model. The proposed model adopted the following components: depthwise-separable convolution, squeeze-and-excitation attention, residual feature propagation, HardSwish activation and global average pooling.

After the training classification accuracy, precision, recall, F1 score, and confusion matrix analysis were used to evaluate the performance of the classification. The computational features were measured in the following ways: parameter count, model size, MACs, FLOPs, inference time and FPS. Then, qualitative explainability analysis was conducted using Grad-CAM and Grad-CAM++. Lastly, the trained proposed model was converted to TensorFlow Lite and seamlessly embedded within the Flutter application for on-device inference.

3.1.2 Proposed Methodology

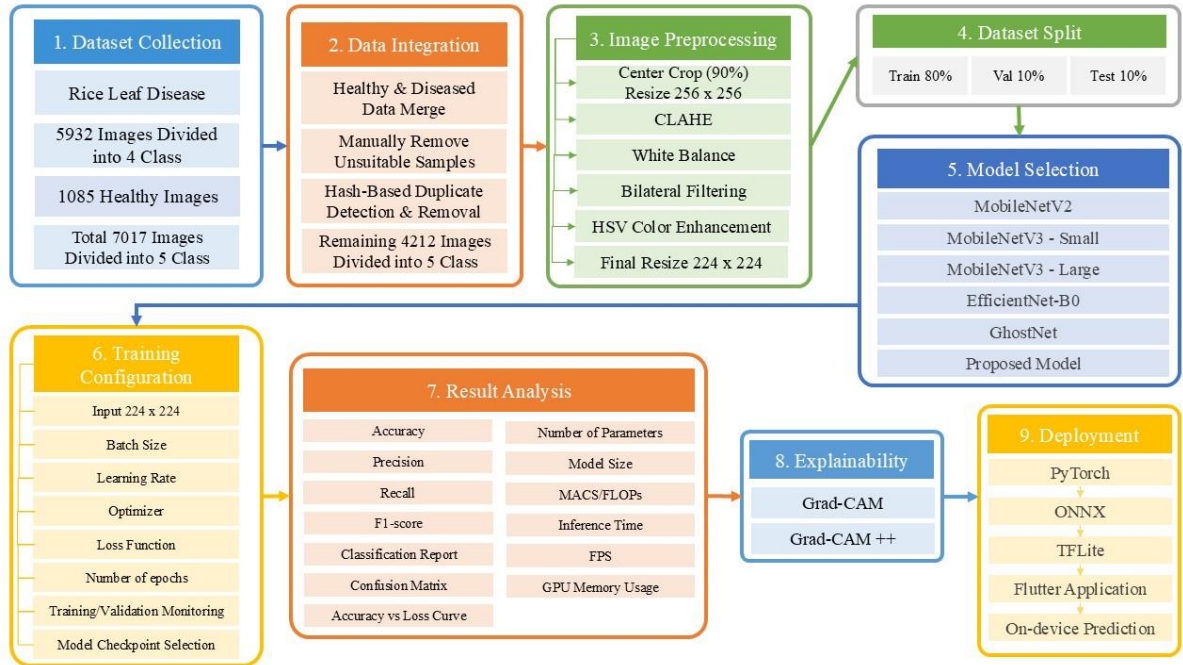


Figure 3.1: Proposed Research Methodology Workflow

The methodology suggested is a sequential research and development process. The first step is dataset collection and integration, where the images of rice leaves available in the dataset are collected, organized, cleaned, and prepared for the next step of processing. Manual quality screening is done to eliminate blurry image or unusable image, and hash-based duplicate detection is done to eliminate duplicate samples.

The images are then passed through the image preprocessing stage to obtain a consistent representation of the images for model development. After preprocessing, the data is divided into three sets: training set, validation set, and test set. The training data is used for model parameter optimization, the validation data is used to observe the training process, and then choose the appropriate training process checkpoint, and the test data is reserved for final independent evaluation.

The model selection stage is considered to established lightweight CNN architectures along with the proposed model. The classification problem is the same for all the selected models. A classification and analysis of the computational performance are provided, and the optimization procedure is defined for training of the proposed model. Then, explainability analysis is carried out with the help of Grad-CAM and Grad-CAM++ to explore spatial regions related to model predictions. Finally, the trained model is converted to TensorFlow Lite format and used to embed in the Flutter mobile app for on-device classification.

3.1.3 Functional and Nonfunctional Requirements

The requirements of the proposed system are divided into functional and non-functional requirements. Functional requirements are the operations the system is expected to perform, whereas non-functional requirements are the expected characteristics of the system as related to performance, efficiency, reliability and deployment.

Table 3.1: Functional Requirements

ID	Requirement	Description
FR1	Image Input	The application shall accept rice leaf images from the camera or gallery.
FR2	Image Preprocessing	The input image shall be processed according to the defined preprocessing pipeline.
FR3	Disease Classification	The system shall classify an image into one of five predefined classes.
FR4	Model Evaluation	The developed models shall be evaluated using classification and computational metrics.
FR5	Result Generation	The application shall generate a classification result from the model output.
FR6	Result Display	The predicted class and associated information shall be displayed to the user.
FR7	On-device Inference	The TensorFlow Lite model shall perform inference locally on the mobile device.

Table 3.2: Non-Functional Requirements

ID	Requirement	Description
NFR1	Accuracy	The system should provide reliable classification performance across the five classes.
NFR2	Computational Efficiency	The model should maintain a relatively low computational requirement.
NFR3	Compactness	The proposed model should have a compact parameter count and model size.
NFR4	Inference Efficiency	The model should provide efficient inference for individual input images.
NFR5	Reliability	The system should produce consistent predictions for comparable input images.
NFR6	Maintainability	The model and application components should be organized in a maintainable manner.
NFR7	Deployment Suitability	The trained model should be convertible to a mobile-compatible format.

3.1.4 Data Flow Diagram

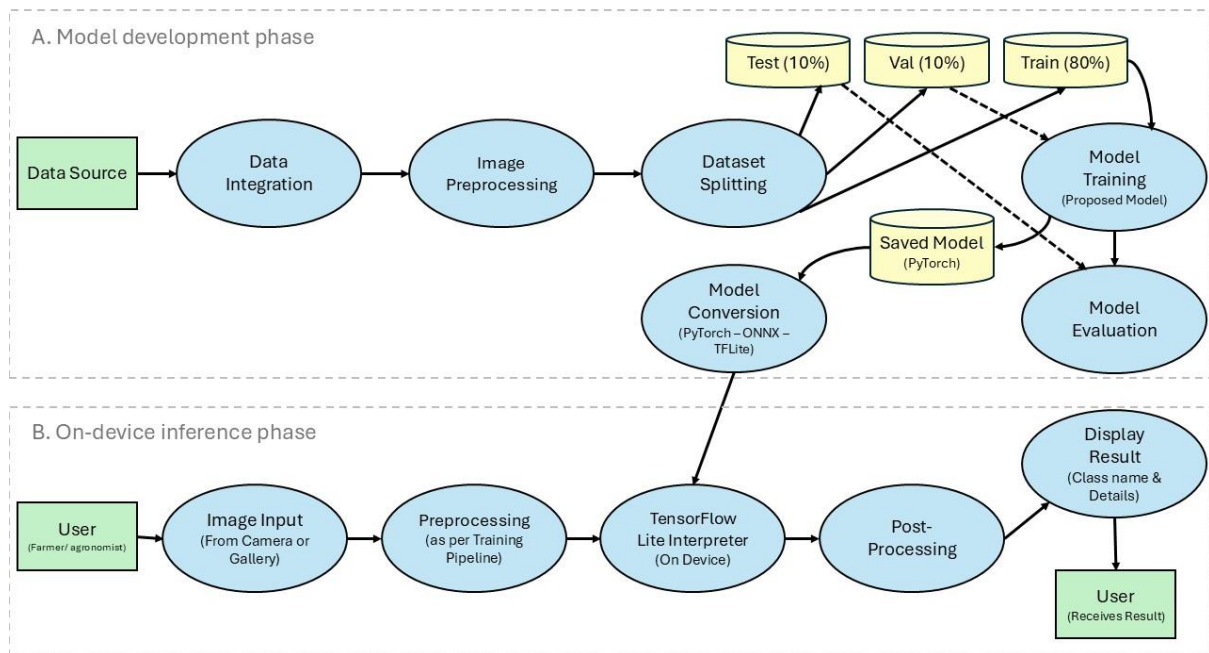


Figure 3.2: Data Flow Diagram of the Proposed System

The Data Flow Diagram represents the movement of data through the model development and mobile inference processes. During model development, rice leaf images are collected and transferred to the data integration stage. The integrated dataset is cleaned and passed through the image preprocessing stage before being divided into training, validation, and testing datasets.

The training and validation data are used during model development, while the test data are retained for independent evaluation. After training and evaluation, the selected trained model is converted into TensorFlow Lite format. The converted model is integrated into the mobile application.

During mobile inference, the user provides an image through the camera or gallery. The application processes the image according to the required input representation and passes it to the TensorFlow Lite interpreter. The model generates prediction scores for the five classes, after which the application processes the output and displays the classification result to the user.

3.1.5 UI Design

Flutter framework has been used to build the mobile app to implement the interface for rice leaf disease classification. The application enables users to upload a picture of rice leaf through the camera of the device or from the gallery.

Once the images are selected, the application preprocesses the image based on the input requirement of the trained model. The TensorFlow Lite interpreter then makes inferences directly on the mobile device. From this prediction, the application processes it and returns the classification result. The interface aims to offer a simple interaction between the user and classification model, such that the inference process of the

classification model could be run without the need of a remote server.

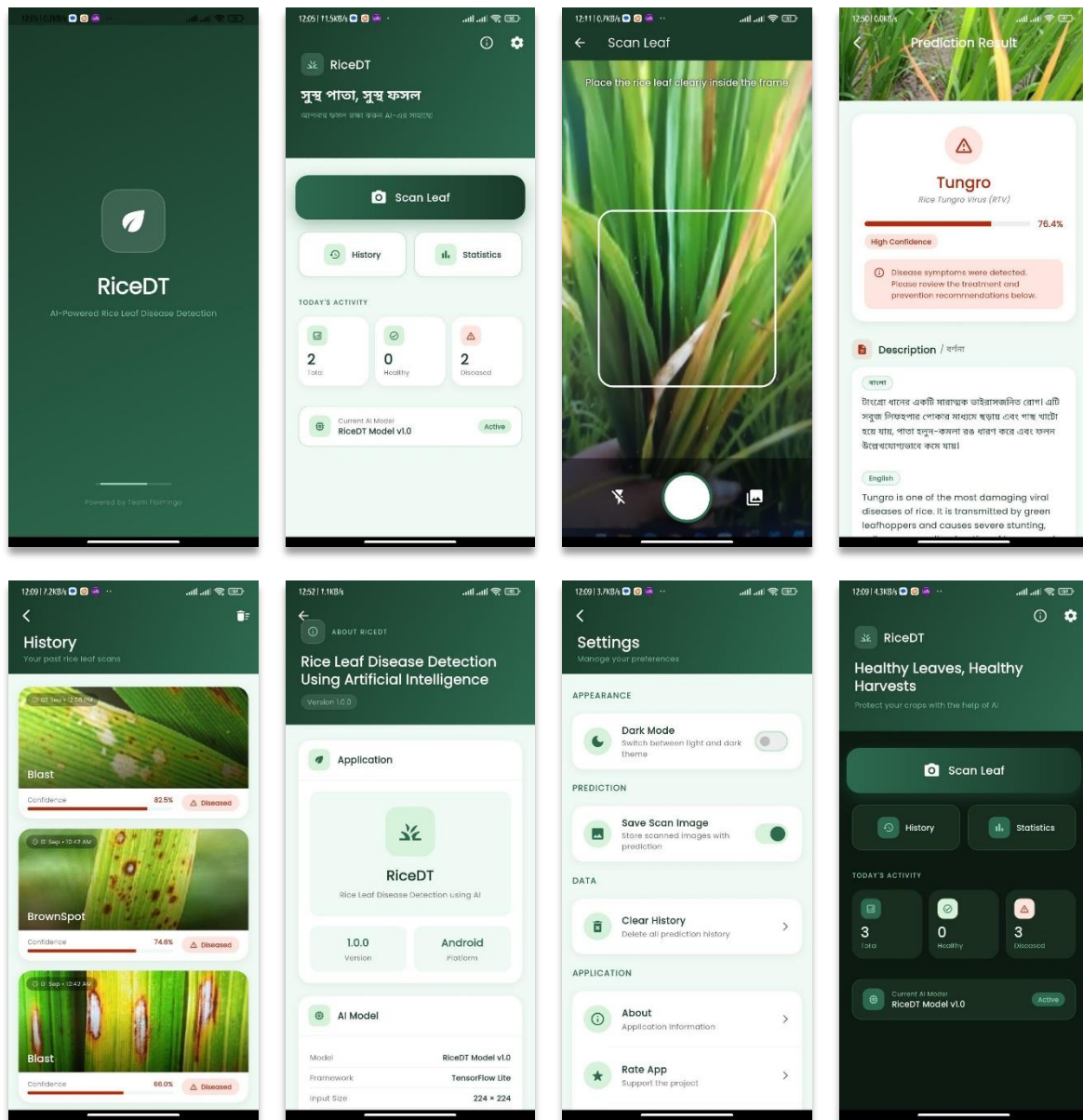


Figure 3.3: User Interface of the Mobile Application

3.2 Detailed Methodology and Design

This section discusses the technical design and implementation process followed to build the proposed system. It discusses the data preparation process, preprocessing pipeline, model architecture, training strategy, evaluation metrics, approach to explainability, and deployment procedure.

3.2.1 Dataset Collection and Integration

The dataset used in this study was created by amalgamating the images collected from two public repositories. Disease-class images were taken from the Rice Leaf Disease Image Samples dataset from P. K. Sethy from Mendeley Data [41]. The dataset consists

of 5,932 images divided into 4 classes of rice leaf disease: Bacterial Blight, Blast, Brown Spot and Tungro.

The images of the Healthy class were downloaded from the Rice Leaf Disease DataSet that was publicly released by M. M. Hasan at Kaggle [42]. The images from both sources were merged and arranged in the 5 classes of interest: Bacterial Blight, Blast, Brown Spot, Healthy and Tungro.

The end result was an integrated dataset of 7,017 images. Images that were blurry and not usable were identified via manual process and discarded to improve the quality and integrity of the data set. Then the Hash-based duplicate detection was used to detect repeated image files. To avoid unnecessary duplication and the potential of duplicate images impacting the subsequent training and evaluation process, duplicate samples were removed.

3.2.2 Final Dataset Distribution

The final data set consists of 4,212 images (five target classes). The data set was split into training, validation, and test sets.

Table 3.3: Final Dataset Distribution

Class	Training	Validation	Testing	Total
Bacterial Blight	656	82	83	821
Blast	621	77	79	777
Brown Spot	536	67	69	672
Healthy	774	96	98	968
Tungro	779	97	98	974
Total	3,366	419	427	4,212

The training subset includes 3,366 images, the validation subset includes 419 images and the testing subset includes 427 images. This distribution is about 80:10:10. The Final Rice Leaf Disease Dataset Distribution for Training, Validation and Testing Sets shown in Figure 3.4.

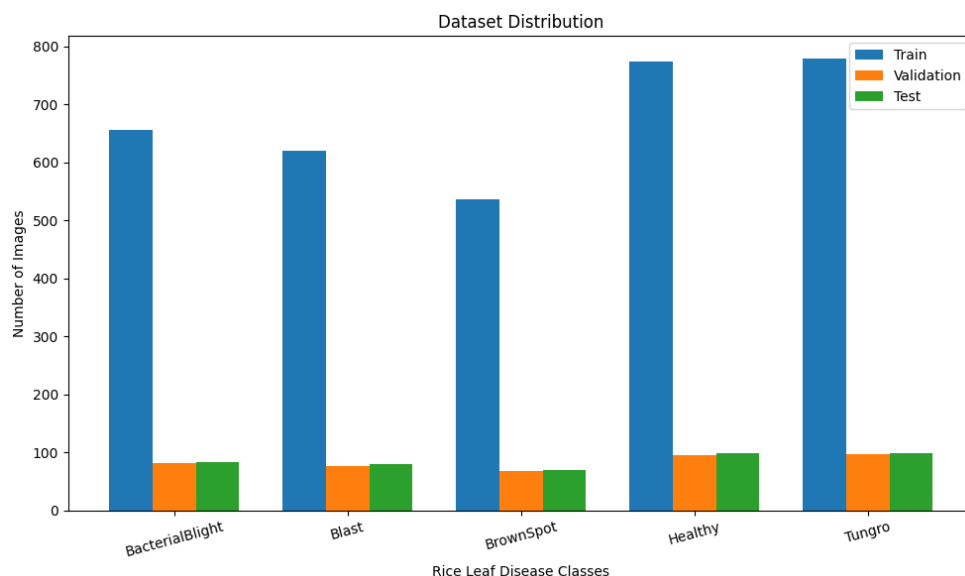


Figure 3.4: Distribution of the Final Rice Leaf Disease Dataset

The class distribution is represented using a bar chart for the three subsets. While class sizes are not equal, each subset contains all five classes, which enable the model to learn and test all of the target classes.

3.2.3 Dataset Splitting and Data Integrity

The cleaned dataset was finally split into training, validation, and testing sets with an approximate ratio of 80:10:10. The training subset was employed to optimize the model parameters, and the validation subset was used during training to check the generalization performance of the model and to choose checkpoints.

Test subset was extracted from the training process and evaluated as a final evaluation of the model. This separation is crucial because optimization of the models might lead to an unrealistic estimate of the generalization performance when using test images.

To ensure that there were no duplicate images within the subsets generated, hash-based verification was also performed. To find potential overlaps between the training data, the validation data and the testing data, pairwise comparison of image hashes was conducted. This process ensures data independence and minimize the risk of outcome being affected by duplicate samples.

3.2.4 Image Preprocessing

To provide a uniform visualization of the rice leaf images, a preprocessing pipeline was designed and implemented using a deterministic approach that was essential to give the CNN models a more uniform and consistent input.

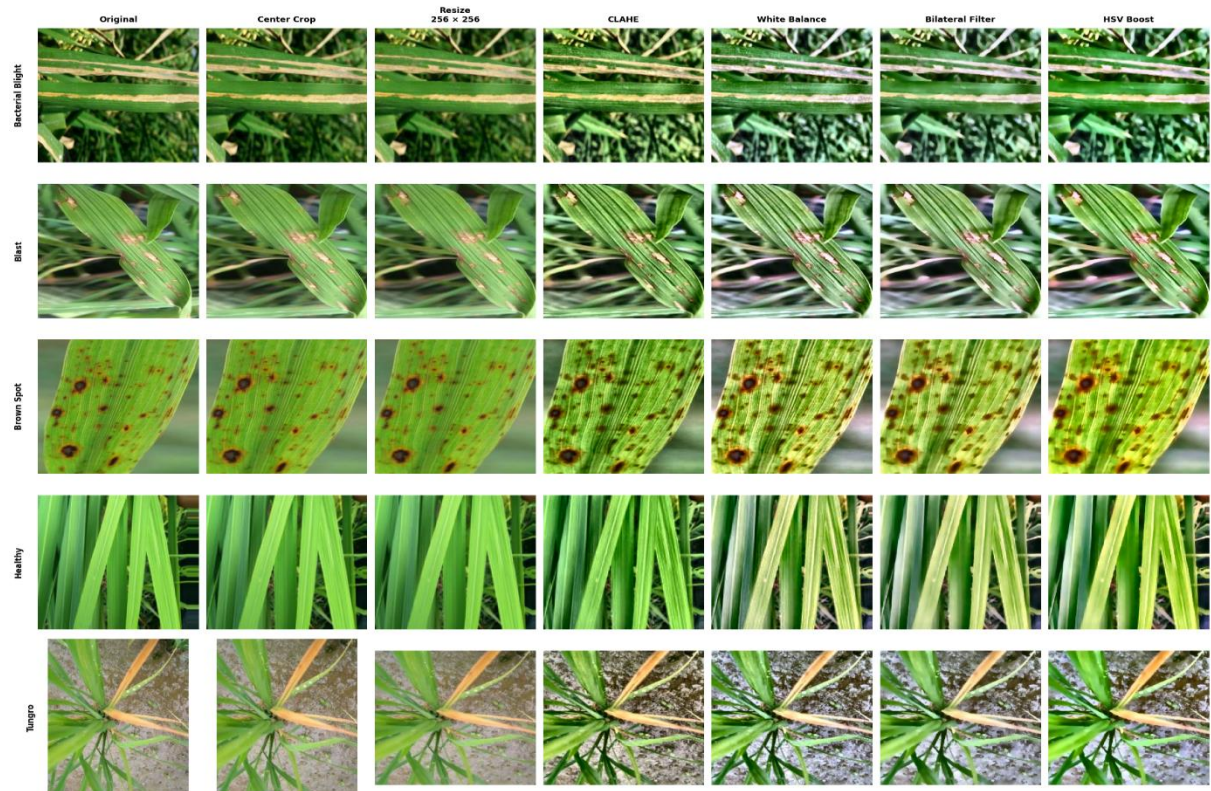


Figure 3.5: Image Preprocessing Pipeline

Center Cropping

The original image is used for a center crop ratio of 90%. The outermost edge is cut out from the image, but the middle part of the image is kept. The operation removes unimportant peripheral content and delivers a more concentrated view of the leaf area.

Initial Resizing

The cropped image is scaled down to 256x256 pixels. This gives an intermediate spatial resolution which remains fixed for subsequent enhancement operations.

CLAHE

Local contrast is enhanced by applying Contrast Limited Adaptive Histogram Equalization (CLAHE). The clip limit is set to 3.0 and there is 8x8 tiling on the luminance component of the LAB color space.

CLAHE is not a global histogram transformation, but a technique that works on local regions. This makes it possible to emphasize the local differences in contrast without overamplifying.

White Balance

White balancing is used to minimize the color differences due to illumination differences. This implementation of the preprocessor uses the 98th percentile of the channel values and scales up to a maximum of 1.5 times.

The intention of this operation is to maintain a similar color appearance of images taken under varying illumination.

Bilateral Filtering

A bilateral filter is used to eliminate noise while preserving significant image edges. The following filtering configuration is being used:

$$d = 9, \sigma_{\text{Color}} = 75, \sigma_{\text{Space}} = 75$$

The bilateral filter considers both spatial distance and intensity similarity when determining the filtering effect. Therefore, smoothing can be done without compromising relatively significant boundaries within the leaf image.

HSV Enhancement

The processed image is then transformed into HSV color space to do controlled enhancement of the colors. The saturation component is multiplied by 1.35, and the brightness component by 1.05.

The aim of this operation is to increase useful color differences in the representation of the leaf, while preserving the global structure of the picture.

Final Resizing and Normalization

Then, the image is resized to the final model input size of 224x224 after the enhancement operations. The image is then translated to a tensor and normalized with the mean and standard deviation of ImageNet.

$$\begin{aligned}\mu &= [0.485, 0.456, 0.406] \\ \sigma &= [0.229, 0.224, 0.225]\end{aligned}$$

For an input pixel value x , normalization can be expressed as:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

Note: x is the normalized value of the input channel, μ is the mean value of the corresponding channel, and σ is the standard deviation of the corresponding channel.

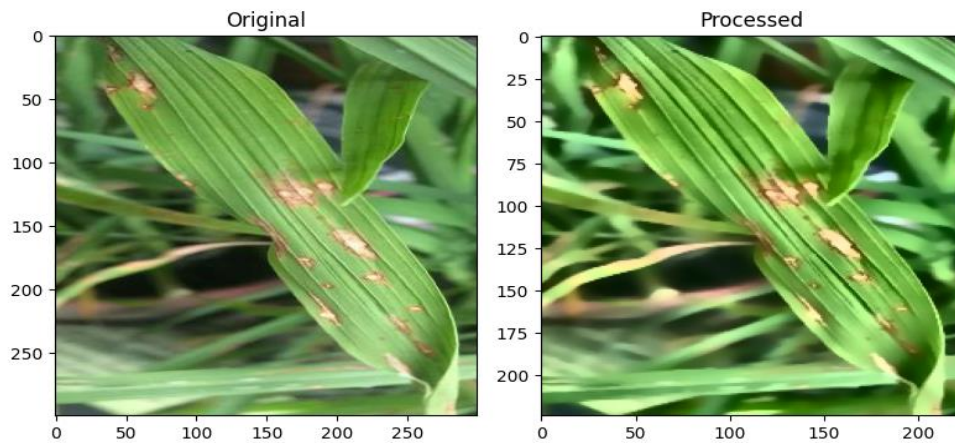


Figure 3.6: Original and Preprocessed Rice Leaf Images

3.2.5 Model Selection

To compare the proposed model with the existing lightweight CNN models, five CNN models are selected as baseline models. These include **MobileNetV2**, **MobileNetV3-Small**, **MobileNetV3-Large**, **EfficientNet-B0** and **GhostNet**.

The baseline models are used as reference models to assess the proposed architecture in terms of classification performance and computational requirements. They were all compared on the same classification problem for leaf disease in rice with a fixed resolution of 224×224 pixels for the input.

The proposed model was designed independently, keeping in mind that it is supposed to be lightweight and is expected to give an optimized feature extraction ability while being efficient as well.

3.2.6 Proposed Model Architecture

The model we propose is designed to be compact: a stem, five Blocks, GAP, dropout and linear classification layer.

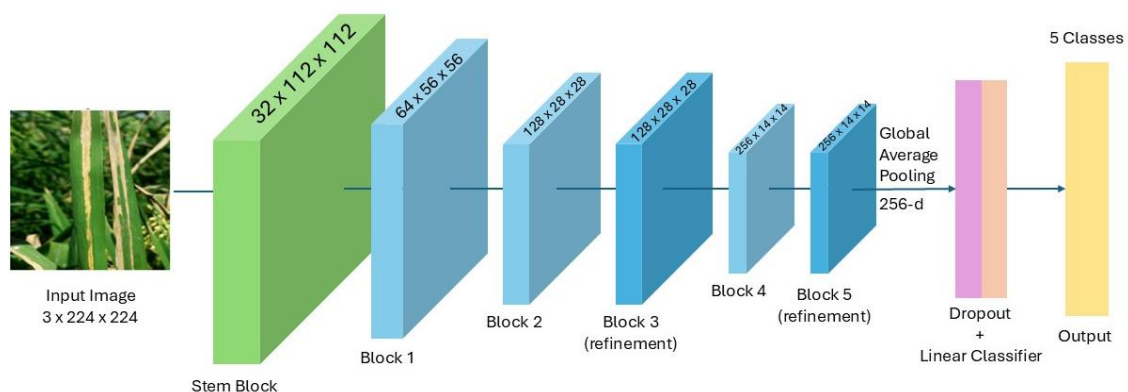


Figure 3.7: Overall Architecture of the Proposed Model

The input to the proposed model is an RGB tensor with dimensions:

$$3 \times 224 \times 224$$

The stem transforms the input into a feature representation of:

$$32 \times 112 \times 112$$

Block 1 upscales the channel dimension from 32 to 64 but downscales the spatial dimension to 56×56 . Block 2 further cuts down on the spatial resolution to 28×28 but expands the number of channels to 128. Block 3 preserves the 128 channel representation and adds further feature refinement.

Block 4 doubles the channel dimension to 128-256 and shrinks the spatial size to 14×14 . Additional refinement is performed on Block 5, which is represented as 256 channels. The output of the resulting feature maps is fed to the global average pooling.

Table 3.4: Proposed Model Architecture

Stage	Channel Transformation	Stride	Output
Input	—	—	$3 \times 224 \times 224$
Stem	$3 \rightarrow 32$	2	$32 \times 112 \times 112$
Block 1	$32 \rightarrow 64$	2	$64 \times 56 \times 56$
Block 2	$64 \rightarrow 128$	2	$128 \times 28 \times 28$
Block 3	$128 \rightarrow 128$	1	$128 \times 28 \times 28$
Block 4	$128 \rightarrow 256$	2	$256 \times 14 \times 14$
Block 5	$256 \rightarrow 256$	1	$256 \times 14 \times 14$
Global Average Pooling	$256 \rightarrow 256$	—	256-d
Dropout	256	—	256-d
Linear Layer	$256 \rightarrow 5$	—	5 classes

3.2.7 Block Design

The proposed model consists of the basic feature extraction unit, Block. Instead of having dedicated, separate figures for the depthwise convolution, global average pooling, and other components, the internal operations are included together in the Block architecture.

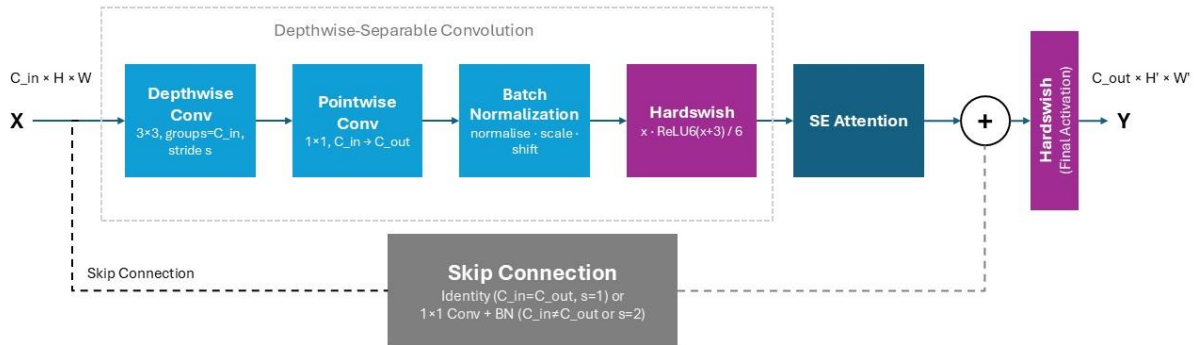


Figure 3.8: Internal Architecture of the Proposed Block

The Block features the depthwise-separable convolution, batch normalization, HardSwish activation, Squeeze-and-Excitation attention, and residual feature

propagation. They have been designed to extract and refine feature representations and have a compact architecture.

The first op is a 3×3 depthwise conv. The depthwise convolution is equivalent to applying one spatial filter per channel to an input of C channels. The depthwise operation may have the number of parameters expressed as:

$$P_{DW} = K^2 C$$

where K is the kernel size and C is the number of input channels.

For a 3×3 kernel:

$$P_{DW} = 9C$$

The depthwise operation is followed by a 1×1 pointwise convolution that combines information across channels. If the number of output channels is C_{out} , the number of pointwise parameters is:

$$P_{PW} = C_{in} C_{out}$$

Therefore, the parameter count of the depthwise-separable convolution, excluding bias and normalization parameters, can be represented as:

$$P_{DS} = K^2 C_{in} + C_{in} C_{out}$$

This decomposition reduces the number of convolutional parameters compared with a conventional convolution, whose parameter count is:

$$P_{Conv} = K^2 C_{in} C_{out}$$

It is followed by HardSwish activation and depthwise / pointwise operations followed by batch normalization. This results in a representation of the feature that is passed in the SE attention mechanism.

Residual connection is used when the dimensions of the input and output are compatible. The original input representation is directly reproduced, and added to the transformed feature representation. The resulting feature representation is then fed into the last HardSwish activation.

Residual feature propagation helps the block to transfer information from previous representations while discriminative features are learnt by the convolutional and attention operations.

3.2.8 Squeeze-and-Excitation Attention within the Block

The proposed Block has a Squeeze-and-Excitation mechanism for channel-wise feature recalibration.

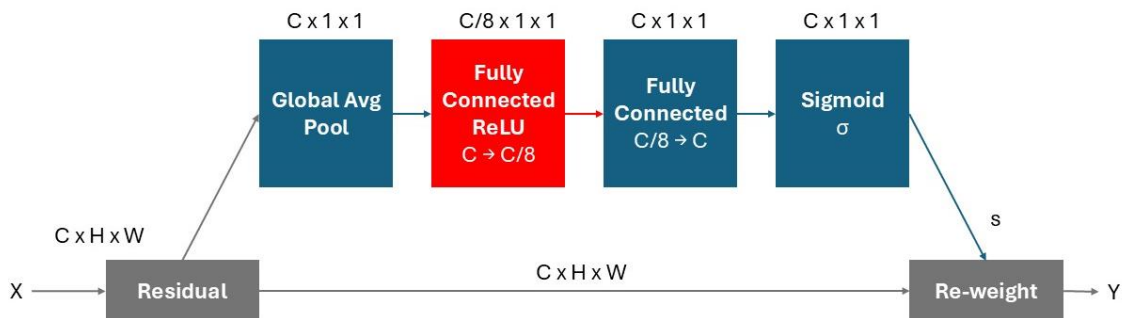


Figure 3.9: Squeeze-and-Excitation Attention Integrated into the Proposed Block

Let the input feature representation be:

$$X \in \mathbb{R}^{C \times H \times W}$$

The squeeze operation applies global average pooling to each channel:

$$z_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W X_c(i, j)$$

where z_c represents the global descriptor of channel c .

The resulting vector is then passed through two fully connected transformations. The first transformation reduces the channel dimension using a reduction ratio of 8:

$$s_1 = \delta(W_1 z)$$

where δ denotes the ReLU activation function.

The second transformation restores the original channel dimension:

$$s_2 = W_2 s_1$$

A sigmoid function is then applied to generate channel-wise attention weights:

$$a = \sigma(s_2)$$

where:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The resulting attention vector is used to recalibrate the feature representation:

$$\hat{X}_c = a_c X_c$$

This enables the model to give different weight to each feature channel before the representation of these channels is fed into the next layer of the Block.

3.2.9 Global Average Pooling and Classification Head

After the five Blocks, the proposed model produces a feature representation of:

$$256 \times 14 \times 14$$

Global Average Pooling is applied across the spatial dimensions. For each channel, the operation can be expressed as:

$$g_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W X_c(i, j)$$

With 256 channels, the operation produces a compact:

$$256\text{-dimensional feature vector}$$

This representation is passed through a dropout layer with a rate of **0.35** and then supplied to a linear classifier. The final classifier produces five output logits:

$$z = [z_1, z_2, z_3, z_4, z_5]$$

corresponding to:

$$\{\text{Bacterial Blight, Blast, Brown Spot, Healthy, Tungro}\}$$

The predicted class can be expressed as:

$$\hat{y} = \arg \max_k z_k$$

where k represents one of the five output classes.

Global Average Pooling does not require a large fully connected layer, which is computing on the full 14×14 spatial representation. This helps to keep the classification head small.

3.2.10 Training Configuration

The proposed model was trained for 60 epochs with a two-stage learning rate schedule for training: warm up phase and cosine learning rate schedule.

In the first 5 epochs, warm-up training was implemented to slowly ramp up the learning rate from 0.01 to 0.05 before the main optimization phase. The highest learning rate was to be:

$$LR_{\max} = 3 \times 10^{-3}$$

and the minimum learning rate was:

$$LR_{\min} = 1 \times 10^{-5}$$

Following the warm up period, CosineAnnealingLR was implemented to gradually decrease the learning rate. The general function that describes the cosine scheduling is:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T}\right)\right)$$

where η_t is the learning rate at step t , $\eta_{\max}$ is the maximum learning rate, $\eta_{\min}$ is the minimum learning rate, and T represents the scheduling period.

The AdamW optimizer was used for parameter optimization with a weight decay of:

$$2 \times 10^{-4}$$

The classification loss was calculated using cross-entropy loss with label smoothing of 0.1. For a multi-class problem, the standard cross-entropy loss can be expressed as:

$$L = - \sum_{k=1}^K y_k \log(p_k)$$

where $K = 5$ is the number of classes, y_k represents the target distribution, and p_k represents the predicted probability for class k .

Label smoothing takes the hard target distribution and replaces it with a softened target distribution, which decreases the certainty only given to one class. The classification head set the drop rate to be 0.35 and the training DataLoader set the batch size to be 32.

Table 3.5: Training Configuration

Parameter	Configuration
Training Strategy	From Scratch
Epochs	60
Warm-up Epochs	5
Optimizer	AdamW
Maximum Learning Rate	0.003
Minimum Learning Rate	0.00001
Weight Decay	0.0002
Scheduler	CosineAnnealingLR
Loss Function	Cross-Entropy Loss
Label Smoothing	0.1

Dropout	0.35
Training Batch Size	32
Input Size	224×224
Number of Classes	5

3.2.11 Classification Performance Evaluation

Confusion matrix analysis and accuracy, precision, recall, and F1 score were used to evaluate the classification performance of the proposed model and baseline models.

Accuracy

Accuracy is the percentage of samples that are correctly classified divided by the number of samples that are tested:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Precision is the ratio of the number of samples correctly classified as a specific class to the total number of samples classified as that class:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

Recall measures the proportion of actual samples of a class that are correctly identified:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score

The F1 score is a harmonic mean of precision and recall:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Confusion Matrix

A confusion matrix was used to examine the classification behavior of the model for each of the five classes. The matrix has the true class on one row and the predicted class on one column, and can be used to highlight correct predictions, as well as class-specific misclassifications.

3.2.12 Computational Efficiency Evaluation

The objective of the proposed model is to be lightweight and mobile, so classification accuracy is not enough to judge the architecture. The performance of classifiers was thus quantified together with their computational properties.

Number of Parameters

The total number of trainable parameters is calculated as:

$$P = \sum_{l=1}^L P_l$$

where P_l represents the number of trainable parameters in layer l .

Generally, the fewer the parameters, the more compact the model representation, and the less storage it will need.

Model Size

The model size is the amount of storage used by the trained model. The approximate storage of the parameters for a model with P parameters, each stored with b bytes, can be represented as:

$$S \approx P \times b$$

The number of models reported would vary depending on the model storage representation being evaluated.

Multiply-Accumulate Operations (MACs)

MACs represent the number of multiply-and-accumulate operations required for an inference pass. For a conventional convolutional layer, the approximate number of MACs can be represented as:

$$\text{MACs} = H_{\text{out}} W_{\text{out}} K^2 C_{\text{in}} C_{\text{out}}$$

where H_{out} and W_{out} represent the output spatial dimensions, K represents the kernel size, and C_{in} and C_{out} represent the input and output channel counts.

Therefore, the approximate computational requirement of a depthwise-separable convolution is:

$$\text{MACs}_{\text{DS}} = H_{\text{out}} W_{\text{out}} (K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}})$$

This computational decomposition is one of the reasons depthwise-separable convolution is commonly used in lightweight CNN architectures.

FLOPs

Floating-point operations (FLOPs) represent the number of arithmetic operations involved in model computation. In the adopted benchmarking convention, FLOPs can be related to MACs approximately as:

$$\text{FLOPs} \approx 2 \times \text{MACs}$$

because one multiply and one addition are generally counted as two floating-point operations.

Inference Time

Inference time measures the time required for the model to process an input sample and produce its prediction. For N measured inference runs, the average inference time is:

$$T_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N T_i$$

where T_i is the measured inference time for run i .

The median inference time is also useful because it is less affected by unusually slow individual measurements.

Frames Per Second

Inference throughput can be represented using frames per second (FPS):

$$\text{FPS} = \frac{1000}{T_{\text{ms}}}$$

where T_{ms} is the inference time in milliseconds.

3.2.13 Standardized Computational Benchmark

To make a fair comparison, a standardized input tensor was used for the computational benchmark:

$$1 \times 3 \times 224 \times 224$$

The batch size of 1 is intentionally different from the batch size of 32 used for training because the inference with a single image is closer to the operating condition of the mobile classification application.

The benchmark consisted of a warmup session of 30 runs, followed by 100 runs of inference timed. The measured properties were: the number of parameters, model size, MACs, FLOPs, peak GPU memory, average inference time, median inference time, standard deviation, minimum inference time, maximum inference time, FPS.

The baseline models and proposed model were both simulated using the same benchmark configuration to make a computational comparison.

3.2.14 Explainability Analysis

Explainability analysis was added to provide qualitative understanding of the image regions related to model predictions. Class-discriminative activation maps were computed using the trained models with Grad-CAM and Grad-CAM++.

3.2.15 Mobile Model Deployment

Once the final trained proposed model was decided upon, it was ready to be integrated into the mobile application. The model conversion process converts the trained model into a format that's appropriate for mobile inference.

The trained model is converted by the following general process:

$$\text{PyTorch} \rightarrow \text{ONNX} \rightarrow \text{TensorFlow Lite}$$

Then the resulting model file is embedded in the Flutter app. The application loads the model using a TensorFlow Lite interpreter and feeds the necessary preprocessed input tensor to the interpreter.

The model is able to receive an image representation that represents:

$$1 \times 224 \times 224 \times 3$$

It is used for mobile inference and produces prediction output for the five target classes. The output scores are then fed into the application, which determines which of the classes is expected to have the highest score.

$$\hat{y} = \arg \max_k p_k$$

where p_k represents the prediction score or probability associated with class k .

The resultant classification is then represented by the user interface of the mobile application. The TensorFlow Lite model is integrated into the application, so the inference process is done directly on the device.

3.3 Project Plan

The project plan outlines the research and development tasks in the project in great stages: literature review and preparation of data sets; development of the model; evaluation of the model; development of the mobile application; documentation.

Table 3.6: Project Plan

Activity	Description
Literature Review	Review existing rice disease classification and lightweight CNN research.
Dataset Collection	Collect and organize rice leaf images.
Dataset Cleaning	Perform manual quality screening and duplicate removal.
Image Preprocessing	Develop and apply the final preprocessing pipeline.
Dataset Splitting	Prepare training, validation, and testing subsets.
Model Selection	Select lightweight baseline CNN architectures.
Proposed Model Development	Design and implement the proposed CNN architecture.
Model Training	Train the proposed and baseline models.
Model Validation	Monitor validation performance and select checkpoints.
Performance Evaluation	Calculate classification metrics and analyze predictions.
Computational Benchmarking	Evaluate model parameters, size, MACs, FLOPs, and inference efficiency.
Explainability Analysis	Perform Grad-CAM and Grad-CAM++ analysis.
Mobile Application Development	Develop the Flutter-based classification application.
Model Conversion	Convert the trained model to ONNX and TensorFlow Lite.
Mobile Deployment	Integrate the TensorFlow Lite model into the application.
Testing and Documentation	Test the complete system and prepare the thesis documentation.

3.4 Task Allocation

This table depicts the timeline of the principal activities in each period of the project from week 12 to week 48.

Table 3.7: Task Allocation

Tasks	Weeks																		
	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48
Data Collection Phase and Integration																			

Data Cleaning and Preprocessing																			
Model Selection and Architecture Design																			
Model Training & Validation																			
Mobile Application Development																			

3.5 Summary

The entire methodology that was applied to develop the proposed light weight CNN based rice leaf disease classification system was presented in this chapter. The research started with a set of 7,017 images, from which blurry images and unusable images were manually filtered out, and hash-based duplicate detection was carried out to reduce the image set. The final dataset consisted of 4212 images of five classes: Bacterial Blight, Blast, Brown Spot, Healthy and Tungro.

Final dataset comprised of 3366 training images, 419 validation images, and 427 testing images. Images were first preprocessed and standardized using a deterministic pipeline before being fed into the model. The preprocessing steps involved were center cropping, resizing, CLAHE, white balancing, bilateral filtering, HSV enhancement, resizing, tensor conversion, and ImageNet normalization.

The proposed model is a compact CNN composed of a stem, 5 Blocks, global average pooling, dropout, and five-class linear classifier. The Block is a convolution layer with the 4 features of depthwise-separable convolution, batch normalization, HardSwish activation, Squeeze-and-Excitation attention, and residual feature propagation.

The proposed model was evaluated against the following models: MobileNetV2, MobileNetV3-Small, MobileNetV3-Large, EfficientNet-B0, and GhostNet. The accuracy, precision, recall, F1-score, and confusion matrix were used to assess the classification performance, and the number of parameters, model size, MACs, FLOPs, inference time, and FPS were used to measure computational efficiency. Grad-CAM and Grad-CAM++ were added to provide qualitative information on the model prediction.

Lastly, the trained proposed model was converted to the TensorFlow Lite format and deployed in the Flutter application to classify the rice leaf disease on the device. In the following chapter, this methodology is used as the basis for the implementation, experimental results, comparative analysis, explainability results and mobile application evaluation.

Chapter 4

Implementation and Results

This chapter focuses on implementing and experimenting the proposed lightweight rice leaf disease classification system. The results are evaluated by analyzing the behavior during training and validation, the classification performance, confusion matrices, computational efficiency, inference performance, explainability, and deployment in mobile devices.

4.1 Environment Setup

A deep learning environment, built in Python, was used to implement the proposed rice leaf disease classification system. Model implementation, training and evaluation was done using PyTorch and mobile deployment was done using TensorFlow Lite (TFLite). The experiments have been performed on Google Colab with GPU acceleration.

A total of 4,212 images from five classes, namely bacterial blight, blast, brown spot, healthy, and tungro were obtained in the final dataset. This data set has 3366 training images, 419 validation images and 427 independent test images. Each of the models evaluated was given an input resolution of 224×224 pixels and had to perform the same five-class classification task.

Table 4.1: Dataset and Experimental Environment Summary

Item	Details
Total Images	4,212
Classes	5
Training Images	3,366
Validation Images	419
Test Images	427
Input Resolution	224×224
Deep Learning Framework	PyTorch
Deployment Format	TensorFlow Lite
Computing Environment	Google Colab / CUDA-enabled GPU

4.2 Testing and Evaluation

The models were evaluated and compared based on their training and validation behavior, test-set classification performance, confusion matrices, computational requirements, and inference performance. A final comparison for classification was performed using an independent test set of 427 images; a standardized batch-size-one was used for computational and inference evaluations.

4.2.1 Models Used for Evaluation

MobileNetV2

MobileNetV2 is a very efficient CNN architecture that is used for image classification. It is a compact structure, which is therefore a suitable ground-truth for lightweight classification systems.

MobileNetV3-Small

A very compact architecture for resource constrained applications. It serves as a reference to compare with the light weight proposed model.

MobileNetV3-Large

MobileNetV3-Large is an upgraded version of MobileNetV3 with a greater capacity. Compares it to a lighter weight, but still relatively large architecture.

EfficientNet-B0:

EfficientNet-B0 is optimized to offer a good trade-off between model size and compute. It was chosen as a baseline CNN model for the classification task, which it was found to be effective.

GhostNet:

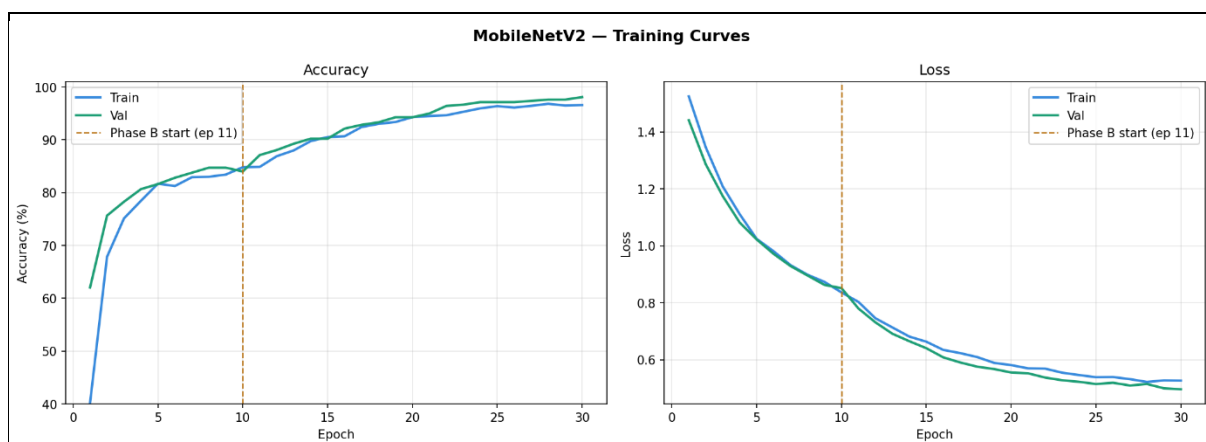
GhostNet is a lightweight architecture to reduce redundant feature generation and computation. It provides an important efficiency-oriented baseline for comparison.

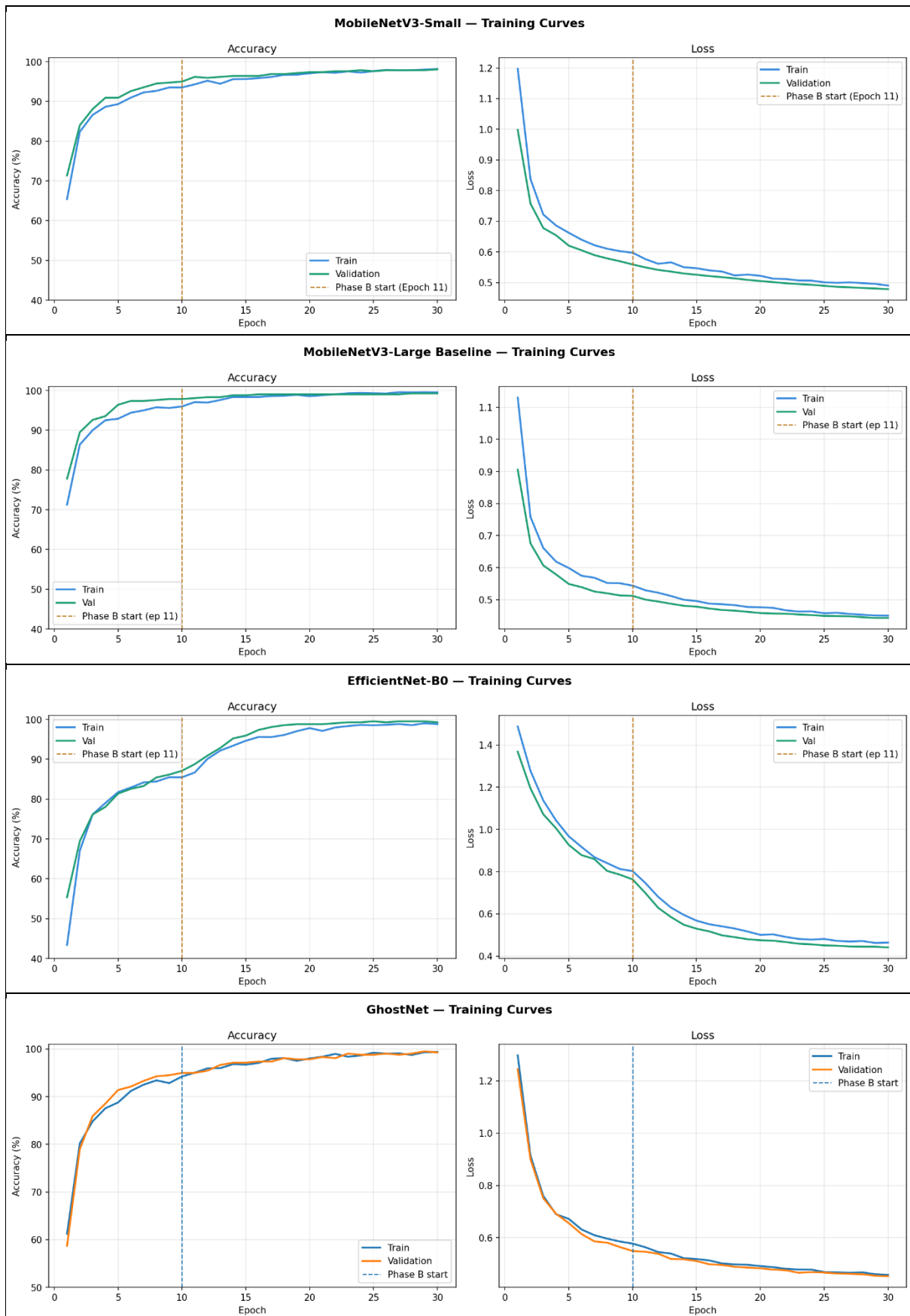
Proposed Model:

The proposed model is the lightweight CNN model designed for rice leaf disease classification. This chapter concentrates on the experimental performance of this building, and how it compares to the baseline models selected, rather than architectural details which are already presented in Chapter 3.

4.2.2 Training and Validation Performance

All six models were evaluated to check the learning behaviour, convergence and validation performance of the models during training and validation. The curves of both accuracy and loss were taken as a measure of a balanced view of the learning process.





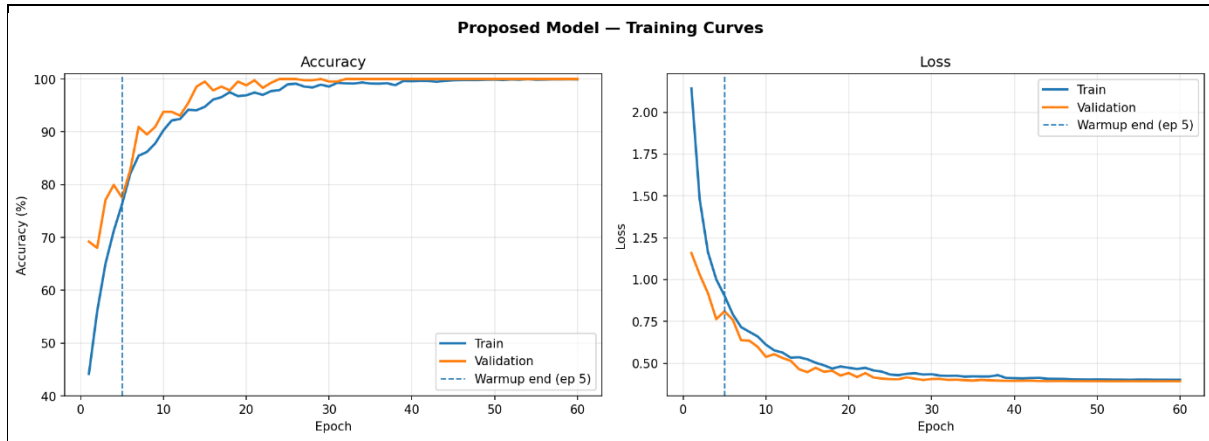


Figure 4.1: Training and validation accuracy vs loss curves of the evaluated models.

The accuracy curves show the learning and validation curves of the different models tested and the loss curves show the behavior of the optimization during training. These curves should be analyzed as a group to investigate the issues of convergence, training vs. validation performance, and learning process stability.

The final interpretation should be made from the actual plots shown. Specifically, the comparison focuses on the models' consistency of convergence, significant differences between training and validation performance, variations between epochs at later stages of training, and stability of the validation performance at the end of training. The results shown below are from an independent test set and constitute the final basis of comparison of the generalization performance.

4.2.3 Classification Performance

The accuracy, precision, recall, and F1-score were used to evaluate the classification performance of the six models assessed on the independent test set. The number of test samples varies between classes, and the overall values of the precision, recall and F1-score are therefore reported using weighted averages.

Table 4.2: Overall Classification Performance

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
MobileNetV2	98.59	98.60	98.59	98.59
MobileNetV3-Small	99.29	99.30	99.29	99.29
MobileNetV3-Large	99.76	99.77	99.76	99.76
EfficientNet-B0	99.76	99.77	99.76	99.76
GhostNet	99.77	99.77	99.77	99.77
Proposed Model	99.77	99.77	99.77	99.77

All six models performed with very high classification performance as shown by the results. MobileNetV2 achieved the minimum test accuracy of 98.59%, while MobileNetV3-Small had a test accuracy of 99.29%. The best accuracy achieved was by MobileNetV3-Large and EfficientNet-B0 with 99.76% accuracy, and GhostNet and the proposed model with 99.77% accuracy.

The proposed model thus obtained the highest test accuracy among all the models observed, which is same as GhostNet. The precision and recall scores were also 99.77%

and 99.65% respectively, with an F1-score of 99.77%, showing that its precision and recall were very consistent across all classifications.

The accuracy of the best models is not very different, but the proposed model has a much different efficiency profile. This is better illustrated by the number of parameters, model size, and measurements of inference provided in the subsequent sections.

4.2.4 Confusion Matrix Analysis

Confusion matrices were prepared for each of the six models to analyze the classification of the correct and incorrect predictions into the five classes of rice leaves.

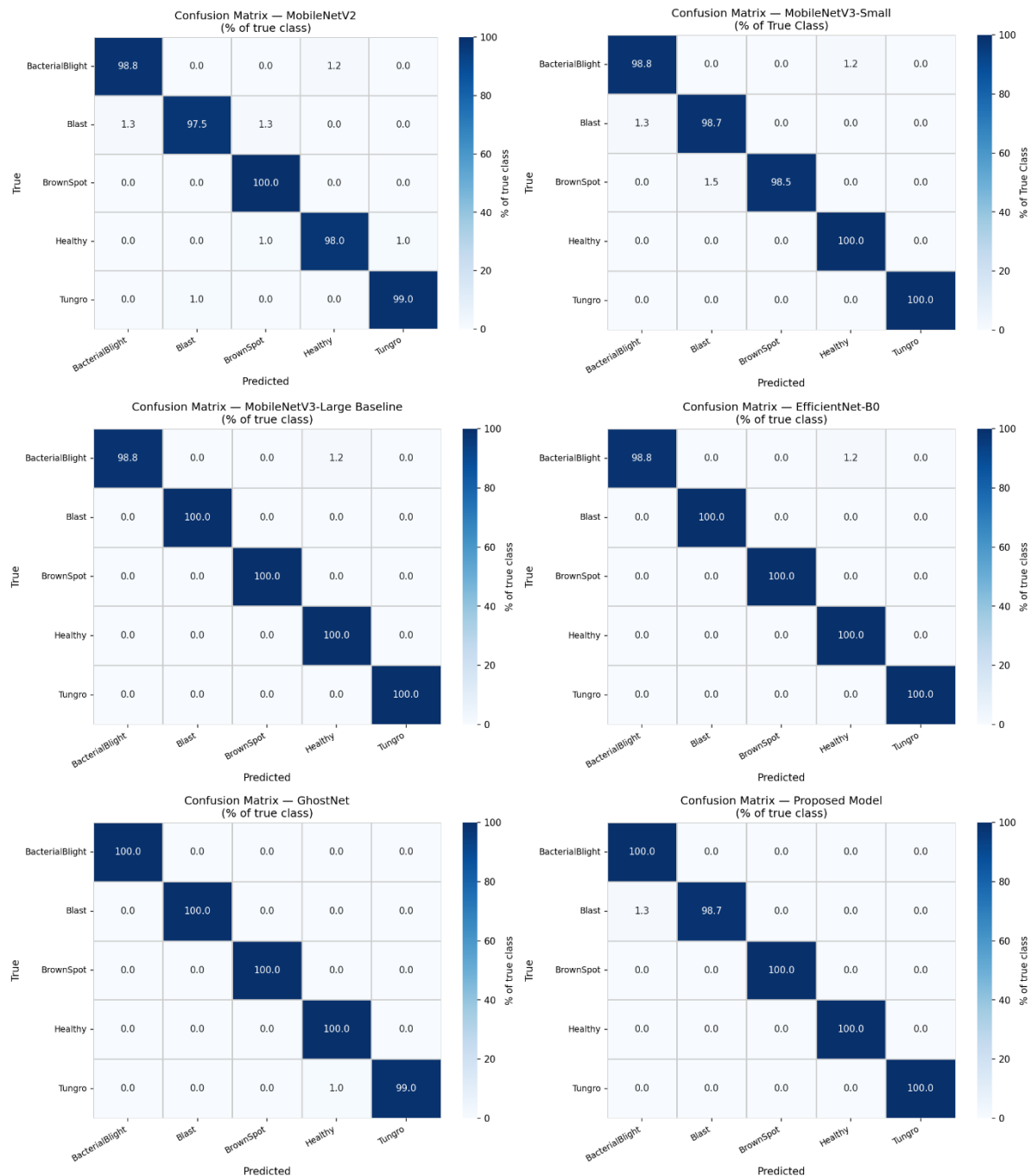


Figure 4.2: Confusion matrices of the evaluated models.

The confusion matrices are a class-level representation of the classification outcomes, and they complement the overall metrics listed in Table 4.2. The matrices are expected to be strongly diagonally dominant, as the test accuracies are very high, and there are only a few off-diagonal predictions.

The confusion matrix of the proposed model corresponds to its test accuracy of 99.77%, meaning that there were only a few misclassified test samples. A direct visual comparison of the six matrices can also be used to assess whether prediction errors are focussed between specific rice disease categories.

4.2.5 Computational Performance

The performance of the six models was compared to one another in terms of the number of parameters, model size, MACs, FLOPs and peak usage of the GPU memory.

Table 4.3: Computational Performance Comparison

Model	Parameters (M)	Model Size (MB)	MACs (M)	FLOPs (M)	Peak GPU Memory (MB)
MobileNetV2	2.230	8.51	326.21	652.43	112.47
MobileNetV3-Small	1.523	5.81	61.46	122.92	101.62
MobileNetV3-Large	4.208	16.05	233.56	467.13	116.90
EfficientNet-B0	4.014	15.31	413.87	827.74	119.29
GhostNet	3.908	14.91	153.88	307.77	130.83
Proposed Model	0.220	0.84	83.61	167.22	129.13

The proposed model has the lowest number of parameters: 0.220 million and the smallest model size of 0.84 MB. The proposed model has about 94.4% fewer parameters and model size than GhostNet, which achieves the same accuracy of 99.77%.

The proposed model needs 83.61 million MACs and 167.22 million FLOPs. The proposed model has significantly fewer parameters and storage than the MobileNetV3-Small model (which has less MACs). The maximum GPU memory proposed for the model was 129.13 MB which is not minimum among the evaluated models. This means that its main computation benefits are not in terms of minimum peak GPU memory but in terms of its extremely low number of parameters and model size.

4.2.6 Inference Performance

The performance of the practical inferences were assessed with a fixed batch size of 1, and the 100 runs were timed after 30 warm up runs.

Table 4.4: Inference Performance Comparison

Model	Average (ms)	Median (ms)	Std. (ms)	Minimum (ms)	Maximum (ms)	FPS
MobileNetV2	5.263	5.021	0.810	4.837	10.093	190.02
MobileNetV3-Small	5.537	5.289	0.862	4.928	10.150	180.60
MobileNetV3-Large	7.750	7.084	2.027	5.693	12.824	129.03
EfficientNet-B0	11.442	10.539	1.741	9.755	18.087	87.40
GhostNet	13.757	13.511	4.167	9.397	29.750	72.69
Proposed Model	2.293	2.211	0.213	2.139	3.552	436.10

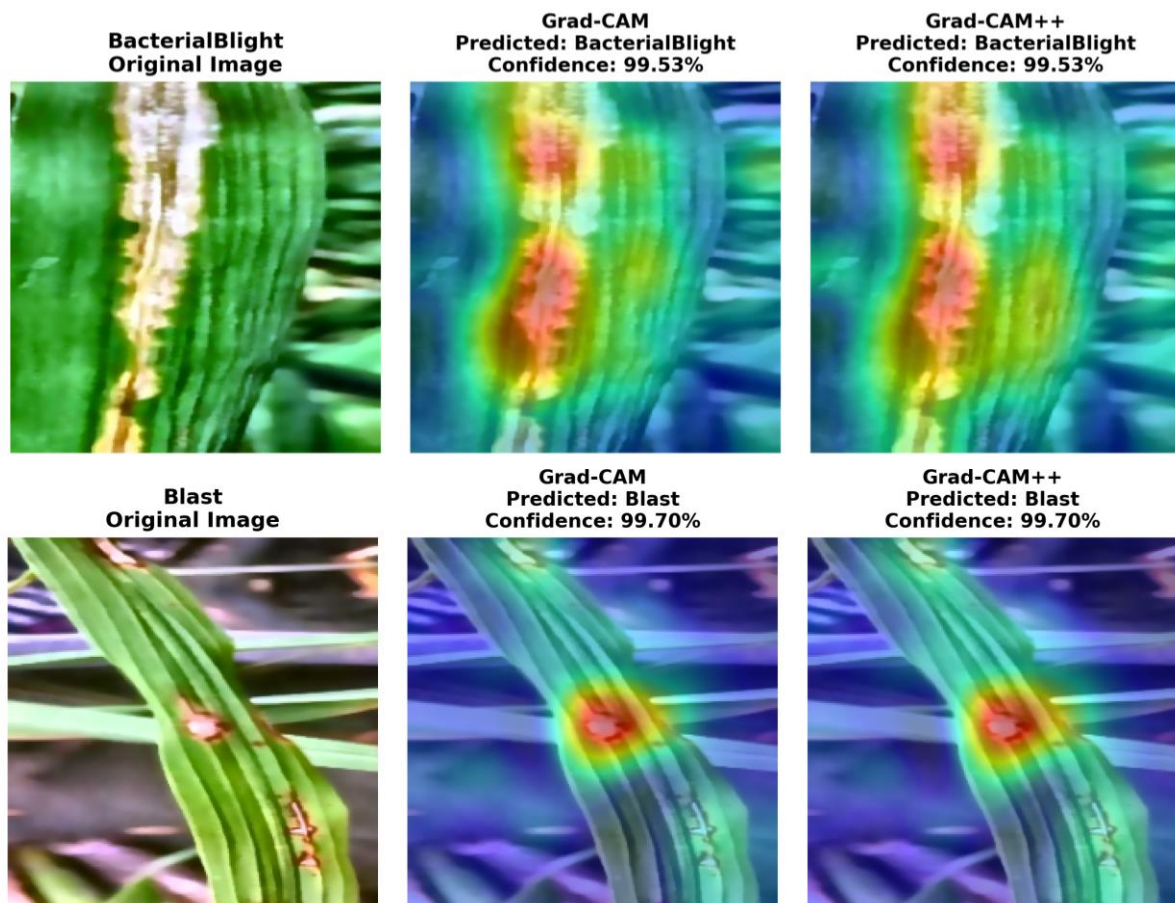
The proposed model gave the minimum average inference time of 2.293ms and the highest throughput of 436.10FPS. It has an average latency of around 56.4% faster than MobileNetV2 and 83.3% faster than GhostNet.

The proposed model also had the lowest standard deviation for inference time of 0.213ms and the measured time was between 2.139 and 3.552ms. This means that the inferences are relatively stable under the benchmark conditions.

4.2.7 Explainability Analysis

A visual interpretation of the proposed model predictions was made using selected images by applying Grad-CAM and Grad-CAM++.

Heatmaps generated show the regions of the image that are important for the model's predictions. These visualizations are used to complement the quantitative classification results and give insight into the spatial basis of individual classifications.



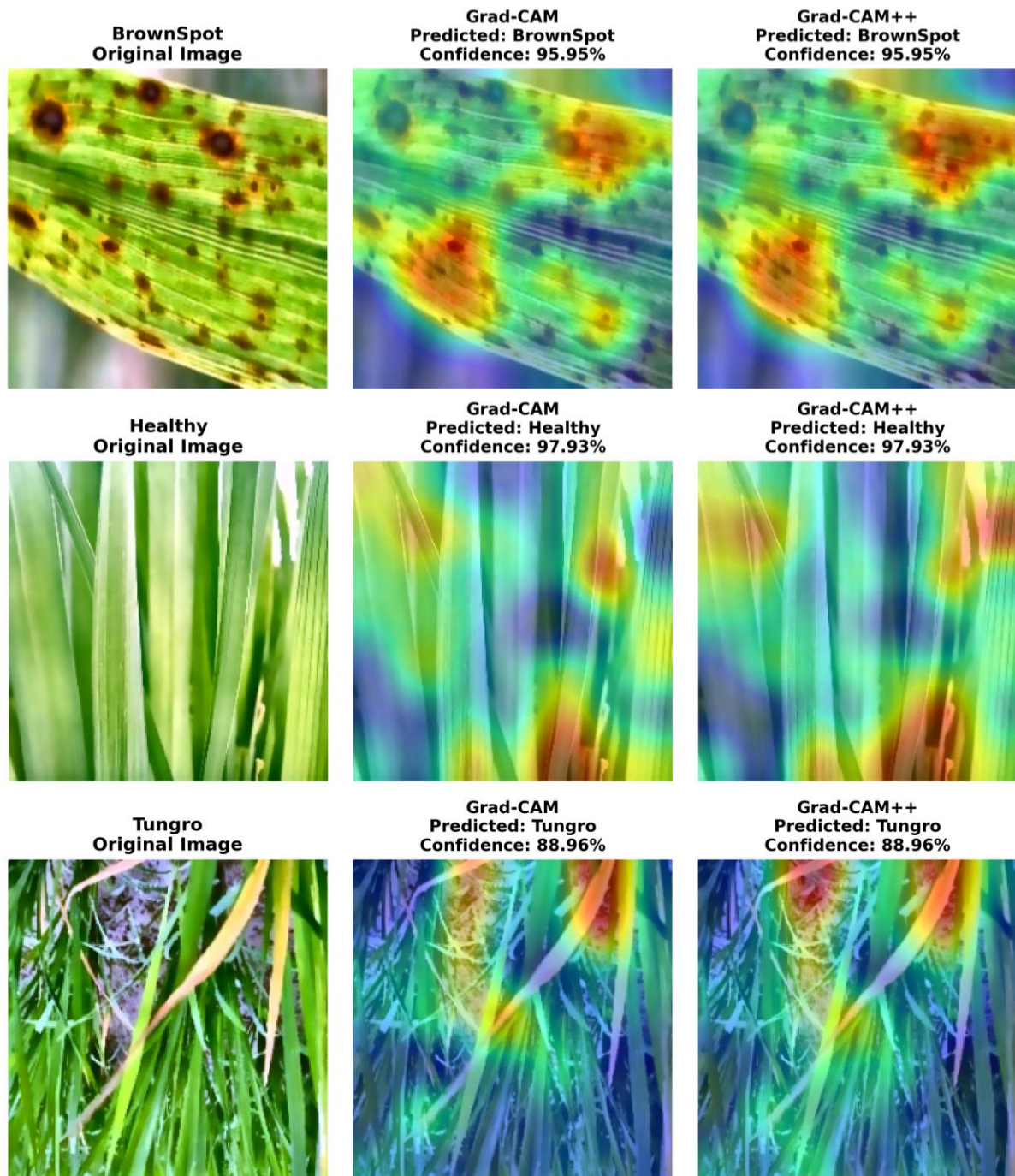


Figure 4.3: Grad-CAM and Grad-CAM++ visualizations of the Proposed Model.

4.2.8 Mobile Deployment

The trained model was then converted to a TensorFlow Lite (TFLite) format and embedded in an Android application, which is based on Flutter, for rice leaf disease classification on device.

The deployment opens the door to the feasibility of the proposed model as an integrated system in a mobile classification system. The compact size of 0.84 MB for the model makes it ideal for mobile applications where storage and computational power might be limited.

The reported 2.293ms inference time and 436.10 FPS is a standardised GPU benchmark and is not to be interpreted as smartphone inference time. The mobile result is not meant to be a mobile-performance benchmark, but rather, it is an indicator of successful model (M) integration and deployment.

4.3 Results and Discussion

Experimental results show that the proposed model offers a good tradeoff between classification accuracy and computational costs. It has performed 99.77% test accuracy, which is the same as GhostNet, among the six models tested for accuracy.

The most striking difference is the compactness of the models. The proposed model requires only 0.220 million parameters and 0.84 MB, significantly less than the models evaluated as baseline models. This is especially interesting since GhostNet managed to reach the same accuracy of 99.77% with 3.908 million parameters and 14.91 MB of storage.

The inference results also confirm the efficiency of the proposed architecture. The model outperformed all the models analyzed with an average latency of 2.293 ms and throughput of 436.10 FPS. It also has a low standard deviation of 0.213ms, which means that it is stable under the benchmark conditions.

The findings also demonstrate that theoretical computational complexity does not always directly map to measured inference speed. The proposed model was found to have more MACs than MobileNetV3-Small, but lower average inference time. Likewise, the proposed model resulted in a larger peak GPU memory footprint than the best models, highlighting the fact that the number of parameters, model storage, computational operations and runtime memory are distinct aspects of model efficiency.

Training and validation curves complement each other in their description of the learning behaviour of the six models, and the confusion matrices offer an overview of the prediction performance of the models on a class level. Furthermore, Grad-CAM and Grad-CAM++ provide qualitative information on the areas that affect the prediction of the proposed model.

Finally, the proposed model is shown to be practical to deploy when it is successfully converted to the TFLite format and integrated with Flutter Android. Overall, The results show that the proposed architecture obtains competitive classification performance and a significantly smaller model size and faster measured inference time than the ones evaluated.

4.4 Summary

The implementation and experimental testing of the proposed rice leaf disease classification system is presented in this chapter. Training and validation curves, test-set classification performance, confusion matrices, computational measurements, inference benchmarking, explainability analysis and mobile deployment were used to evaluate six models.

The proposed model had an accuracy of 99.77% on the test set, which was in line with GhostNet which obtained the highest classification accuracy. Meanwhile, it needed only 0.220 million parameters, 0.84 MB of storage with the lowest measured inference latency of 2.293 ms and the highest throughput of 436.10 FPS.

Additional visual interpretability was provided by the Grad-CAM and Grad-CAM++ analyses and the feasibility of mobile deployment was shown through TFLite conversion and Flutter Android integration. As a whole, the results demonstrate the good accuracy–efficiency tradeoff of the proposed model in the task of lightweight rice leaf disease classification.

Proposed CNN Model: A lightweight CNN that has been designed specifically for a rice leaf disease classification. It is compared with the chosen baseline architecture with respect to classification accuracy and computational efficiency.

Chapter 5

Engineering Standards and Design Challenges

This chapter explores the engineering standards that were taken into account during the design of the proposed rice leaf disease classification system and how this system has ethical and social, as well as environmental and sustainability implications. It also shows the project management and financial aspects and projects the project to complex engineering problem and engineering activity characteristics.

5.1 Compliance with the Standards

The implementation of the proposed rice leaf disease classification system required software development, experimentation of machine learning, data management, and deployment of mobile applications. Accordingly, software quality, coding, data management, mobile software and computing environments were considered in the development process, along with standards, and existing engineering practices.

5.1.1 Software Standards

PEP 8 – Python Coding Style

The coding convention was considered for implementation using python, was PEP 8. It gives style guidelines for code formatting, naming, indenting, organizing and readability. Another way to do the coding is to adhere to a project specific coding style that offers more flexibility but could result in less uniformity among modules. It was preferred to use PEP 8 as the project heavily uses the Python language to implement the deep learning algorithms, and a standardized coding style contributes to better maintainability and readability.

ISO/IEC 25010 – Software Quality Model

Software quality characteristics like performance efficiency, reliability, maintainability, and portability are assessed by ISO/IEC 25010. This was of special interest, since the developed system was tested on accuracy for classification, besides the size of the model and the performance of the inference. It would be easier to consider only functional correctness and accuracy; this would offer less comprehensive software quality consideration. The framework presented in ISO/IEC 25010 is then useful in thinking about the quality of the system being developed as a whole, in terms of its broader quality characteristics.

TensorFlow Lite/Android Compatibility

The trained model was then converted into TensorFlow Lite format. The other possibility is to do inference remotely which might allow more complex models to be supported but would also require network dependency, communication delay and added infrastructure. TFLite-based on-device inference was chosen for executing inferences locally and maintaining consistency with the lightweight nature of proposed model.

5.1.2 Hardware Standards

It was essential for the project to have computing hardware that could run deep learning model training and evaluation. During the experiments, GPU acceleration was used as the CNN training requires a large volume of parallel numerical computation.

The CUDA environment of a GPU was utilized for development and testing. A viable alternative would be CPU-only computation that would lead to lower hardware requirements but much larger model training and computational evaluation time. GPU acceleration was therefore chosen as this allows the workload to be executed more efficiently.

The system is intended for deployment on Android mobile devices, not necessarily on dedicated high-performance devices. This strategy minimizes the equipment needed by end users and enables the system to be used in resource-limited scenarios.

5.1.3 Communication Standards

The proposed classification application is mainly done locally on Android device. Thus, there is no need for continuous network communications for the essential disease classification task.

An alternative approach is to send images remotely to an inference server via an Internet connection. One possibility is to send images to a remote inference server over the Internet. While this might enable centralized updates of the model and more complex computations, it also would create a reliance on network availability, network latency, server infrastructure, and extra data-privacy concerns.

Consequently, on-device inference with the TFLite model was chosen, which enables the core classification process to happen without a requirement for data to be continually transferred between the device and an external server. This design is especially suited for agricultural settings, where it may not always be possible to obtain reliable Internet connectivity.

5.2 Impact on Society, Environment and Sustainability

The proposed system is designed to facilitate the identification of rice leaf diseases by automatic image classification. As an important agricultural crop, a lightweight disease-detection tool can help users to detect disease symptoms earlier and aid the agriculture decision making process. The system should be used as a decision support tool and never be used as a substitute for professional agricultural diagnosis at the same time.

5.2.1 Impact on Life

Rice disease may have a negative impact on crop health and productivity. A computer vision-based classification system can be used to automatically identify the common rice leaf diseases from images, and can assist users in faster identification of potentially

affected plants.

By enabling users to interact with the classification system via an Android device, the mobile-oriented design can further enhance accessibility. This could be helpful for farmers, students, agriculture practitioners, and researchers who need a portable disease classifier.

But with wrong predictions, wrong agricultural decisions will be taken. Hence, the system should be viewed as a supporting tool and the suggestions it gives as a complement to agricultural know-how and when needed, expertise.

5.2.2 Impact on Society & Environment

The system can have a social benefit due to easier access to agricultural image analysis systems, which are automatic. A mobile version may alleviate the need for high-performance computing infrastructure at the field, and could also make computer-vision-based support for agriculture more available.

Early diagnosis of disease could also help inform more focused agricultural interventions from an environmental point of view. But the system developed is not directly linked to pesticide doses or treatment decisions. So, any statements about pesticide use or environmental pollution reduction should not be made if it hasn't been validated in the field.

The small size also decreases the size of the model, enabling inference on mobile devices. This can help to decrease reliance on continuous cloud-based processing and associated infrastructure.

5.2.3 Ethical Aspects

The development of the system was done with the consideration of several ethical issues. The classifier should not be offered as a diagnostic tool that doesn't make mistakes since predictions made by machine learning can be incorrect, especially when images are very different from the ones used to train the model.

Another key factor to consider is the quality of the datasets. The training data should be a proper representation of the disease classes, and wrongly categorized or labelled images can impact the reliability of the model. The system should thus be able to communicate predictions responsibly, without making uncertain predictions a diagnosis.

Consider also privacy when users take and upload photos using a mobile app. The images don't have to be sent continuously to the remote server for the heart of the classification process, as the proposed application is performing on-device inference with the TFLite model. This will decrease the amount of external image transmission during normal inference.

5.2.4 Sustainability Plan

Computational efficiency was considered as an important aspect while designing the proposed system. The developed model is compact, has 0.220 million parameters, and its

model size is 0.84MB, which can be embedded in a mobile application with relatively small storage requirements.

The on-device inference also helps decrease reliance on an always-on cloud service for simple classification. The future versions can be further optimized to improve sustainability by optimizing the model, quantizing, inference on mobile devices and evaluating on lower power edge devices.

Furthermore, to ensure long-term sustainability, the dataset must be maintained and the model updated to incorporate new disease patterns, environmental conditions, or new rice varieties that should be represented.

5.3 Project Management and Financial Analysis

The project was implemented in a series of phases, including data preparation, model development, training, verification, model explainability, and deployment on mobile devices. The development process focused on the completion and validation of the core classification model, rather than using the model in the mobile app.

The project was mainly software based with software tools and computing resources already available. Thus, computational resources, development of hardware, software development, testing and deployment became the significant considerations of cost.

Table 5.1: Estimated Project Budget

Item	Estimated Cost (BDT)
Development Computer/Laptop	80,000
GPU/Cloud Computing Resources	10,000
Internet and Data	5,000
Android Testing Device	20,000
Software and Development Tools	0
Dataset Preparation and Processing	5,000
Miscellaneous/Testing Expenses	5,000
Estimated Total	125,000

The above is an estimated development budget and not costs of the finished academic project. The project has made use of several software frameworks and software development environments that are not purchased outright.

Alternative Budget

An alternative development scheme would use an existing PC, cloud based access to GPUs (if available) and an existing Android smartphone for testing.

Table 5.2: Alternative Low-cost Budget

Item	Estimated Cost (BDT)
Existing Computer	0
Available Cloud GPU Resources	0
Internet and Data	5,000
Existing Android Device	0
Software and Development Tools	0
Dataset Preparation	3,000

Miscellaneous	2,000
Estimated Total	10,000

The alternative budget is significantly lower since it assumes the primary hardware, and mobile test device are already present. One method is good for an academic prototype as opposed to buying dedicated hardware, where the objective is to keep development costs down.

Revenue Model

The developed system is mainly for academic prototype hence there was not a major concern of generating direct commercial income. The potential revenue sources for such a development as a feasible and practical agricultural application might be a free-basic classification service with premium optional features, institutional and agricultural organisation licences, and agricultural service provider partnerships.

A basic service, which would be free of charge or free of cost, would make the service more accessible, and premium service may offer extra functionality like disease history, farm level monitoring or advanced agricultural decision-support tools. Before being used as a production agricultural service, further testing and validation of the technology would have to be done in the field before any commercial deployment.

5.4 Complex Engineering Problem

Implementing a pre-defined machine-learning algorithm is not enough for the development of the rice leaf disease classification system. Several challenges had to be overcome in integrating the data preparation, image classification, lightweight model design, computational evaluation, explainability, and mobile deployment of the project, along with accuracy, memory, and storage constraints, while also meeting usability requirements.

There are thus several interacting engineering requirements and constraints in the problem. The project is mapped to the characteristics of a complex engineering problem as outlined by the project framework in the following sections.

5.4.1 Complex Problem Solving

Table 5.3: Mapping with Complex Engineering Problem.

EP1 Dept of Knowledge	EP2 Range Of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent Of Stake- holder Involvement	EP7 Interdependence
✓	✓	✓	✓			✓

Several aspects of computer science and engineering were essential to the development of classification system for rice diseases. The project consisted of pre-processing and processing the images of the rice leaves, training and evaluating various CNN

classification models, designing a lightweight architecture, and analyzing their classification and computational performance. The proposed model was optimized for both accuracy and efficiency: to run on mobile deployment, a low number of parameters, a small storage size and fast inference were desirable.

The project also included a systematic evaluation of trained models such as accuracy, precision, recall, F1-score, confusion matrices, computational cost, inference latency, and methods to explain the models like Grad-CAM and Grad-CAM++. Deep learning methods that have been developed were introduced to the task at hand of rice leaf disease classification, while the visual features of the five rice leaf disease classes were taken into account.

The various phases of the project were clearly interrelated. These three issues—dataset preparation, model architecture, and computational efficiency—impacted model training, the model architecture impacted computational requirements, and computational efficiency impacted the feasibility of deployment on Android. The trained model was then ported to TFLite and deployed in a mobile application as a part of the overall system.

The project was not significantly based on formal engineering codes or a lot of consultation with stakeholders. The development was primarily addressed in terms of technical implementation and experimental evaluation and mobile deployment, instead of formal stakeholder study or regulatory engineering requirements.

Mapping with Knowledge Profile

Table 5.2: Mapping with knowledge Profile.

K1 Natural Science	K2 Mathematics	K3 Engineering Fundamentals	K4 Specialist Knowledge	K5 Engineering Design	K6 Engineering Practice	K7 Comprehension	K8 Research Literature
✓	✓	✓	✓	✓	✓	✓	✓

The project integrated knowledge and characteristics of rice leaf diseases with mathematical, computational, and engineering concepts. Mathematical principles were used to optimize models, classify the models, to use probability in models, and to evaluate the performance of the models; the engineering fundamentals were used to process data, implement algorithms, and build systems.

The deep learning, CNN architectures, computer vision, lightweight model design and explainability techniques were used throughout the project. This knowledge has been utilized in designing the proposed lightweight model and comparing it with the other lightweight architectures. Engineering design was used to determine how well the classification technology performs while accounting for computational efficiency and mobile deployment needs.

The project also incorporates hands-on engineering aspects like model implementation, training, testing, computational benchmarking, Grad-CAM and Grad-CAM++ analysis, model conversion to TFLite, and integration with Android. Furthermore, the existing

CNN architectures and agriculture disease classification was taken into account in the process of development and evaluation of the system.

5.4.2 Engineering Activities

The project also involved multiple engineering activities related to resource selection, system interaction, innovation, societal consequences, and the application of engineering knowledge.

Mapping with Complex Engineering Activities

Table 5.3: Mapping with Complex Engineering Activities.

EA1 Range of resources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
✓	✓	✓	✓	✓

This system development needed various resources, such as the rice leaf image dataset, Google Colab with GPU resources, deep learning frameworks (Python), software libraries, visualization tools, TFLite, and Flutter for creating mobile applications. The resources were applied throughout the various phases of model development, evaluation and deployment.

The various components of the project included dataset preparation, model development, training and evaluation, analysis of explainability, and mobile deployment. The most important engineering contribution was the development of a lightweight CNN architecture designed for the rice leaf disease classification task, while focusing on high classification accuracy while simultaneously ensuring low computational and storage costs.

The project also took into account the potential implications of automating disease diagnosis in agriculture. Inaccurate predictions can affect agricultural decisions, therefore the system is not used as a diagnostic tool, but as a decision support. In general, various Deep Learning and software development methods were used to solve a real-world problem in the agricultural sector and assembled into an end-to-end mobile classification system.

5.5 Summary

In this chapter, the engineering standards, design considerations, societal impacts, financial aspects and complex engineering characteristics of the proposed rice leaf disease classification system have been discussed. The software, hardware, and communication aspects were considered, focusing on maintaining software, efficiency, and on-device mobile inference.

The project also took its potential impact on those who use the land for agriculture, society as a whole, the environment and ethics into account. The proposed system was designed as a light weight decision support system and the financial analysis showed a conventional system development budget and a budget based on existing resources that would result in a lower-cost system.

Lastly, the project was mapped to the complex engineering problem and engineering activity characteristics. The mapping showed that multidisciplinary knowledge was used throughout the development of the system, as well as conflicting accuracy-efficiency requirements, analytical evaluation, stakeholder considerations, resource coordination, innovation, and interdependent engineering activities.

Chapter 6

Conclusion

The proposed rice leaf disease classification system is developed and summarized in this chapter to complete the system development. It also provides some comments on the current system and suggests potential avenues for future system enhancement.

6.1 Summary

The project aimed to develop an image-based rice leaf disease classification system for classification of rice leaf images into 5 classes: Bacterial Blight, Blast, Brown Spot, Healthy and Tungro. This system was designed with the goal of having high classification performance and being a computationally efficient system for mobile deployment.

A lightweight CNN architecture was introduced in this paper by performing depth-wise separable convolution, Squeeze and Excitation attention, residual feature propagation, global average pooling, and a compact classification head. Google Colab GPU was used to train and test the model. Multiple models with a good track record of their lightweight nature, like MobileNetV2, MobileNetV3-Small, MobileNetV3-Large, EfficientNet-B0 and GhostNet, were also tested for comparison.

From the experimental results, the proposed model has an accuracy of 99.77% and precision, recall and F1 score of 99.77%. Simultaneously only 0.220 million parameters were present in the model and its model size was about 0.84 MB. An average of 2.293ms was recorded for the inference time which approximately translates into 436.10 FPS in the experimental setup. The results illustrate that the proposed architecture has good classification accuracy and computational efficiency.

Confusion matrices and Grad-CAM and Grad-CAM++ visualizations were also used to investigate model predictions and regions of the image used for classification, respectively. Finally, the trained model was converted to TensorFlow Lite and embedded into an Android application to show the feasibility of deployment of the rice leaf disease classification system on a mobile platform.

Overall, the project was a success in building and testing a lightweight rice leaf disease classification model and showcasing its implementation by a mobile app.

6.2 Limitation

The proposed system has a good performance on the test data set, however, there are several limitations.

First, the quality and variety of data in the model greatly affect its performance. The images taken under varying light, background, camera quality and rice variety and disease stages may vary from the training images and may impact the classification.

Second, the system only works for the five disease/condition classes represented in the data set. The current model cannot define other Rice Diseases or rare condition. Likewise, the system does not identify the stage or the severity of a disease, but just classifies an image.

Third, the experiments were carried out mainly with the data and the computing facilities available. The model was implemented in an Android app, but more real-world testing in various agricultural contexts and with various mobile devices was not conducted.

A further restriction is the fact that a classification model can make an incorrect prediction. Thus, the system could be regarded as a support tool for agriculture experts or a professional diagnosis system, not a substitute for either. The visual explanations of model attention by Grad-CAM and Grad-CAM++ do not ensure that the model's explanation is biologically sound.

Lastly, the results of the computations and inferences made have been obtained in the current set up of the experiment conducted for the project. The actual performance can change from smartphone to smartphone depending on its hardware, operating system, device specifications and deployment configuration.

6.3 Future Work

There are a number of ways this system could be improved in the future. The dataset can be extended by obtaining larger and more diverse set of rice leaf images under different lighting, background, disease, rice variety and geographical settings. This might enhance the strength and applicability of the classification model.

There can be further updates in future versions: addition of other rice diseases, disease severity classification, and would enable the system to give more detailed information about the disease status of a rice plant rather than just the five classes that are already present.

Optimizing the proposed model further could be investigated for mobile and edge devices. Other techniques like model quantization, pruning, and hardware-specific optimizations can be explored to further decrease the computational load without compromising the classification accuracy.

Another significant line of research is the use of the tools in the real world. Field test of the application on various Android cell phones and direct images from the rice field would give a true idea of its performance in the field.

Lastly, other agricultural support functions, such as the prediction history, the monitoring of images and other information for agricultural decision-making, could be added to the mobile application. Any such extensions would need to be further validated to make sure the system is consistently reliable and useful for real-world agricultural applications.

References

- [1] P. Rawat, A. Pandey, and A K, "Rice Leaf Diseases Classification Using Deep Learning Techniques," 2023 International Conference on Networking and Communications (ICNWC), Apr. 2023, doi: <https://10.1109/icnwc57852.2023.10127315>.
- [2] Mehedi Hasan Bijoy et al., "Towards Sustainable Agriculture: A Novel Approach for Rice Leaf Disease Detection Using dCNN and Enhanced Dataset," IEEE Access, vol. 12, pp. 1–1, Jan. 2024, doi: <https://10.1109/access.2024.3371511>.
- [3] M. R. Mahin, M. R. Ahmed, Most. S. Islam, A. R. Samad, M. M. Hasan, and A. A. Habib, "A Lightweight Deep Learning Architecture for Multi-Crop Disease Detection Using Custom CNN and EfficientNet-B0," 2026 IEEE 2nd International Conference on Quantum Photonics, Artificial Intelligence & Networking (QPAIN), pp. 1–6, Apr. 2026, doi: <https://10.1109/qpain69676.2026.11545965>.
- [4] C. Pan, S. Wang, Y. Wang, and C. Liu, "SSD-YOLO: A Lightweight Network for Rice Leaf Disease Detection," Frontiers in Plant Science, vol. 16, Aug. 2025, doi: <https://10.3389/fpls.2025.1643096>.
- [5] U. Sirisha, K. Sharma, P. Praveen, D. Parashar, and N. S. K. M. K. Tirumanadham, "A Lightweight Residual Dilated CNN–Transformer Framework for Efficient Rice Leaf Disease Classification," Research Square (Preprint), Mar. 2026, doi: <https://10.21203/rs.3.rs-9038735/v1>.
- [6] Q. Jing, "A Lightweight Deep Learning Model for Crop Disease Detection on Mobile Devices," International Journal on Smart Sensing and Intelligent Systems, vol. 19, no. 1, Jan. 2026, doi: <https://10.2478/ijssis-2026-0047>.
- [7] S. Bhagat et al., "Advancing Real-Time Plant Disease Detection: A Lightweight Deep Learning Approach and Novel Dataset for Pigeon Pea Crop," Smart Agricultural Technology, vol. 7, p. 100408, Mar. 2024, doi: <https://10.1016/j.atech.2024.100408>.
- [8] C. R. Carlson, J. Alghazo, and W. U. Hasan, "AI-Powered Detection of Rice Leaf Diseases: A Survey of CNN Architectures and Attention-Based Methods," 2025 IEEE International Conference on Imaging Systems and Techniques (IST), pp. 1–6, Oct. 2025, doi: <https://10.1109/ist66504.2025.11268365>.
- [9] R. Kayne, H. Heart, S. C. Brillo, K. E. Mata, and J. C. Morano, "An Enhancement of CNN Algorithm for Rice Leaf Disease Image Classification in Mobile Applications," arXiv.org. Accessed: Sep. 01, 2026. [Online]. Available: <https://arxiv.org/abs/2412.07182>
- [10] M. Tasci, "DBLA-MobileNetV2: Real-Time Rice Leaf Disease Detection on Edge Devices With Dual-Branch Lightweight Attention," IEEE Access, vol. 14, pp. 22356–22371, 2026, doi: <https://10.1109/access.2026.3662799>.
- [11] P. Pai et al., "Deep Learning-Based Automatic Diagnosis of Rice Leaf Diseases Using Ensemble CNN Models," Scientific Reports, vol. 15, no. 1, Jul. 2025, doi: <https://doi.org/10.1038/s41598-025-13079-z>
- [12] M. Abdullah et al., "Deep Learning-Based Rice Leaf Disease Detection Using Custom CNN Models," Journal of Computing & Biomedical Informatics, vol. 9, no. 02, 2023, Accessed: Sep. 01, 2026. [Online]. Available: <https://www.jcbi.org/index.php/main/article/view/1090>
- [13] M. N. Velasco, P. Egar, K. N. Cristobal, V. P. Manalang, L. Cydea, and C. Paul, "Deep Learning-Driven Mobile Application for Rice Crop Disease Detection Using CNN," 2025 International Conference on Biomimetic Intelligence and Robotics (ICBIR), pp. 454–459, May 2025, doi: <https://10.1109/icbir65229.2025.11162996>.
- [14] M. Zhang et al., "Dual-Attention-Enhanced MobileViT Network: A Lightweight Model for Rice Disease Identification in Field-Captured Images," Agriculture, vol. 15, no. 6, pp. 571–571, Mar. 2025, doi: <https://10.3390/agriculture15060571>.
- [15] O. O. Martins, C. C. Oosthuizen, and D. A. Desai, "<Scp>RiceLeafClassifier</Scp>-V1.0: A Quantized Deep Learning Model for Automated Rice Leaf Disease Detection and Edge Deployment," Engineering Reports, vol. 7, no. 6, Jun. 2025, doi:

<https://doi.org/10.1002/eng2.70231>

- [16] B. Gan, G. Pu, W. Xing, L. Wang, and S. Liang, "Enhanced YOLOv8 With Lightweight and Efficient Detection Head for Detecting Rice Leaf Diseases," *Scientific Reports*, vol. 15, no. 1, Jul. 2025, doi: <https://doi.org/10.1038/s41598-025-06843-8>
- [17] J. Zheng et al., "FPGA-Based Low-Power High-Performance CNN Accelerator Integrating DIST for Rice Leaf Disease Classification," *Electronics*, vol. 14, no. 9, p. 1704, Apr. 2025, doi: <https://doi.org/10.3390/electronics14091704>
- [18] P. Zhang, R. Li, Y. Liu, G. Sun, and C. Wen, "I-GhostNetV3: A Lightweight Deep Learning Framework for Vision-Sensor-Based Rice Leaf Disease Detection in Smart Agriculture," *Sensors*, vol. 26, no. 3, p. 1025, Feb. 2026, doi: <https://doi.org/10.3390/s26031025>
- [19] J. Li et al., "Lightweight Rice Leaf Spot Segmentation Model Based on Improved DeepLabv3+," *Frontiers in Plant Science*, vol. 16, Aug. 2025, doi: <https://doi.org/10.3389/fpls.2025.1635302>
- [20] Y. Xu, D. Li, C. Li, Z. Yuan, and Z. Dai, "LiSA-MobileNetV2: An Extremely Lightweight Deep Learning Model With Swish Activation and Attention Mechanism for Accurate Rice Disease Classification," *Frontiers in Plant Science*, vol. 16, Aug. 2025, doi: <https://doi.org/10.3389/fpls.2025.1619365>
- [21] J. Cui, Y. Yang, Z. Chen, S. Hao, and F. Tan, "LMRNet: A Lightweight Convolutional Neural Network for Real-Time Mountain Rice Leaf Disease Recognition on Edge Devices," *Frontiers in Plant Science*, vol. 17, Mar. 2026, doi: <https://doi.org/10.3389/fpls.2026.1789523>
- [22] Poningsih, Suhendra, and Ahmad Zamsuri, "MACHINE LEARNING WITH LIGHTWEIGHT CNN (RESNET-18) for EARLY DETECTION of RICE LEAF DISEASES," *JITK (Jurnal Ilmu Pengetahuan dan Teknologi Komputer)*, vol. 11, no. 4, pp. 1083–1096, May 2026, doi: <https://doi.org/10.33480/jitk.v11i4.7358>
- [23] Azis, "Welcome to Zscaler Directory Authentication," Doi.org. Accessed: Sep. 01, 2026. [Online]. Available: <https://doi.org/10.52436/1.jutif.2025.6.3.4452>
- [24] Zhang, X. Hao, J. Liang, L. Li, and W. Li, "A Lightweight and Generalizable Deep Learning Framework for Early Detection of Rice Leaf Diseases in Complex Field Environments," *Scientific Reports*, vol. 15, no. 1, Dec. 2025, doi: <https://doi.org/10.1038/s41598-025-32568-9>
- [25] H. K. Kondaveeti and C. G. Simhadri, "Evaluation of Deep Learning Models Using Explainable AI With Qualitative and Quantitative Analysis for Rice Leaf Disease Detection," *Scientific Reports*, vol. 15, no. 1, Aug. 2025, doi: <https://doi.org/10.1038/s41598-025-14306-3>
- [26] S. Duhan, P. Gulia, N. S. Gill, and E. Narwal, "RTR_Lite_MobileNetV2: A Lightweight and Efficient Model for Plant Disease Detection and Classification," *Current Plant Biology*, vol. 42, p. 100459, Jun. 2025, doi: <https://doi.org/10.1016/j.cpb.2025.100459>
- [27] R. Hartono, Nanang Maulana Yoeseph, Fendi Aji Purnomo, S. Alim, and Agus Purnomo, "A Lightweight Convolutional Neural Network for Rice Leaf Disease Detection Integrated in an Android Application," *Bulletin of Electrical Engineering and Informatics*, vol. 14, no. 4, pp. 2705–2720, Aug. 2025, doi: <https://doi.org/10.11591/eei.v14i4.9260>
- [28] Santosh Kumar Upadhyay and Rajesh Prasad, "Efficient-ViT B0Net: A High-Performance Light Weight Transformer for Rice Leaf Disease Recognition and Classification," *Journal of the Nigerian Society of Physical Sciences*, vol. 7, no. 4, p. 2940, Nov. 2025, doi: <https://doi.org/10.46481/jnsps.2025.2940>
- [29] T. W. Utami, M. Novita, and K. Latifa, "Implementation and Comparative Analysis of CNN and Transfer Learning Models (EfficientNetB0, MobileNetV2, and ResNet50) for Rice Leaf Disease Detection Based on Digital Images," *Journal of Applied Informatics and Computing*, vol. 9, no. 6, pp. 3862–3870, Dec. 2025, doi: <https://doi.org/10.30871/jaic.v9i6.11616>
- [30] X. Yuan et al., "RiceDetect-Net: A Lightweight Real-Time Detection Framework for Rice Diseases," *BMC Plant Biology*, vol. 26, no. 1, Feb. 2026, doi: <https://doi.org/10.1186/s12870-026-08341-z>
- [31] H. Liu et al., "Efficient Field-Scale Detection System of Intensive Rice Lesions on Edge Devices," *Information Processing in Agriculture*, Feb. 2026, doi: <https://doi.org/10.1016/j.inpa.2026.02.008>

- [32] U. V. Nnamdi and V. Abolghasemi, "Optimised MobileNet for Very Lightweight and Accurate Plant Leaf Disease Detection," *Scientific Reports*, vol. 15, no. 1, Dec. 2025, doi: <https://doi.org/10.1038/s41598-025-27393-z>
- [33] W. Pi, T. Zhang, R. Wang, Z. Zhou, G. Ma, and Y. Wang, "LightWaveNet: A Lightweight Wavelet-Enhanced High-Low-Frequency-Aware Network With Multi-Stage Supervision for Rice Disease Recognition," *Frontiers in Plant Science*, vol. 17, Jan. 2026, doi: <https://doi.org/10.3389/fpls.2026.1692649>
- [34] S. Barman et al., "Optimized Crop Disease Identification in Bangladesh: A Deep Learning and SVM Hybrid Model for Rice, Potato, and Corn," *Journal of Imaging*, vol. 10, no. 8, pp. 183–183, Jul. 2024, doi: <https://doi.org/10.3390/jimaging10080183>
- [35] Y. Ibrahim, M. B. Mu'azu, M. A. Jimoh, and A. Yunus, "Mobile-Optimised Deep Learning Architecture for Multi-Crop Disease Detection Using CNN and SVM," *Engineering Headway*, vol. 37, pp. 87–97, Mar. 2026, doi: <https://doi.org/10.4028/p-hanb9o>
- [36] P. Li, J. Zhou, H. Sun, and J. Zeng, "RDRM-YOLO: A High-Accuracy and Lightweight Rice Disease Detection Model for Complex Field Environments Based on Improved YOLOv5," *Agriculture*, vol. 15, no. 5, pp. 479–479, Feb. 2025, doi: <https://doi.org/10.3390/agriculture15050479>
- [37] Z. Li et al., "Rice Disease Detection: TLI-YOLO Innovative Approach for Enhanced Detection and Mobile Compatibility," *Sensors*, vol. 25, no. 8, p. 2494, Apr. 2025, doi: <https://doi.org/10.3390/s25082494>
- [38] Y. Zhang and J. Yu, "Enhanced MobileNet-V2 Model With Adaptive Attention and Cross-Layer Fusion for Fine-Grained Identification of Rice Diseases and Pests for Sustainable Agriculture," *Discover Applied Sciences*, vol. 8, no. 2, Dec. 2025, doi: <https://doi.org/10.1007/s42452-025-08198-1>
- [39] Chew Jing Xan, Hermawan Nugroho, Sivaraman Eswaran, and Tay Fei Siang, "Effective Edge Solution for Early Detection of Rice Disease on ARM- M Microcontroller," *IEEE Access*, vol. 12, pp. 1–1, Jan. 2024, doi: <https://doi.org/10.1109/access.2024.3470970>
- [40] R. Deng et al., "Automatic Diagnosis of Rice Diseases Using Deep Learning," *Frontiers in Plant Science*, vol. 12, Aug. 2021, doi: <https://doi.org/10.3389/fpls.2021.701038>
- [41] Prabira Kumar sethy, "Rice Leaf Disease Image Samples," *Mendeley Data*, vol. 2, Jul. 2024, doi: <https://doi.org/10.17632/fwcj7stb8r.2>
- [42] M. M. Hasan, "Rice Leaf Disease DataSet," *Kaggle.com*, 2023. <https://www.kaggle.com/datasets/mdmehedihasan19/rice-leaf-disease-dataset> (accessed Apr. 09, 2026)
- [43] nazibur74, 'GitHub - nazibur74/RiceDT-FYDP', *GitHub*. Accessed: Sep. 04, 2026. [Online]. Available: <https://github.com/nazibur74/RiceDT-FYDP>